VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnanasangama, Macche, Santibastwada Road, Belagavi–590018, Karnataka

---

A

UG BE PROJECT REPORT

(PROJECT COURSE CODE : BIC685)

SEVENTH SEMESTER BE UG – PHASE II

on

AutoPark

A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers

Submitted in partial fulfillment of the requirement for the degree of

Bachelor of Engineering

in

Computer Science & Engineering – CSE (IC)

by

USN : 1RR23IC___	[Student Name 1]
USN : 1RR23IC___	[Student Name 2]
USN : 1RR23IC___	[Student Name 3]
USN : 1RR23IC___	[Student Name 4]

Under the guidance of

[Name of the Project Guide]

Project Guide

Professor, Dept. of CSE (IC)

Rajarajeswari College of Engineering, Bangalore, Karnataka

Department of Computer Science & Engineering

(IoT, Cyber Security with Blockchain Technology)

(An Autonomous College affiliated to VTU Belgaum, accredited by NBA & NAAC)

14, Ramohalli Cross, Mysore Road, Kumbalgodu Post,

Bengaluru–560074, Karnataka, India

2026–27

November 2026

CERTIFICATE

Certified that the major project work, Phase–2 (BIC685) entitled **"AutoPark: A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers"** carried out by [Student Name 1] (USN 1RR23IC___), [Student Name 2] (USN 1RR23IC___), [Student Name 3] (USN 1RR23IC___) and [Student Name 4] (USN 1RR23IC___), are bonafide students of the CSE Dept. (IC) of Rajarajeswari College of Engineering, Bangalore, Karnataka, India in partial fulfillment for the award of Bachelor of Engineering in Computer Science & Engineering (IC) of the Visvesvaraya Technological University, Belagavi, Karnataka for the VII Semester autonomous course during the academic year 2026–27 and is being guided by [Name of the Project Guide] (Project Guide). It is certified that all corrections / suggestions indicated for the major project work of the Phase–2 have been incorporated in the 7th sem project report submitted to the CSE (IC) department through the Project Coordinator & HoD.

This Phase–2 project report has been approved as it satisfies the academic requirement in respect of project work of the said 7th sem prescribed for the said degree for the VTU batch.

Project Guide Sign
Name : [Name of the Project Guide]

Project Coordinator Sign
Name : [Name of the Project Coordinator]

HoD Sign
Name : [Name of the Head of the Department]

Principal Sign
Name : Dr. R. Balakrishna

External VII Sem Major-Project Viva-Voce (SEE)

Name of the project examiners with date :

1 : (Int) Signature : _____

2 : (Ext) Signature : _____

STUDENTS DECLARATION

Certified that the 7th Sem Phase–2 project work entitled **"AutoPark: A Camera-Based Smart Parking System with Automated Number Plate Recognition and Micro-Actuated Boom Barriers"** with the course code BIC685 (6 Credits, 100 Marks–CIE, 100 Marks–SEE) is a bonafide work that was carried out by ourselves in partial fulfillment for the award of degree of Bachelor of Engineering in Computer Science & Engg (IC) of the Rajarajeswari College of Engineering, affiliated to the Visvesvaraya Technological University, Belagavi, Karnataka during the academic year 2026–27 for the VII Semester Autonomous Course. We, the students of the 7th sem Phase–2 project group / batch no. B–__ do hereby declare that the entire main-project has been done on our own and we have not copied or duplicated any other's work. The results embedded in this project report has not been submitted elsewhere for the award of any type of degree and is fully genuine and novel work.

[Student Name 1]
Sign
USN : 1RR23IC___

[Student Name 2]
Sign
USN : 1RR23IC___

[Student Name 3]
Sign
USN : 1RR23IC___

[Student Name 4]
Sign
USN : 1RR23IC___

Date : /11/2026
Place : Bengaluru – 74

ACKNOWLEDGEMENT

We sincerely thank all the people who have directly or indirectly helped us in the successful completion of the 7th Sem CSE (IC) Under-Graduate BE project work. We thank all the following members whole-heartedly for all sorts of help rendered to us.

1. Chairman RRGI – Dr. A. C. Shanmugam
2. Vice Chairman RRGI – Dr. A. C. S. Arun Kumar
3. Executive Director RRGI – Dr. S. Vijayanand
4. Special Officer RRGI – Dr. S. Jeyabalan
5. Principal – Dr. R. Balakrishna
6. Registrar – Prof. Savitha
7. Rector – Dr. Mariappan
8. Department HOD – [Name of the Head of the Department]
9. Project Guide – [Name of the Project Guide]
10. Dean (Academics) – Dr. L. Rangaiah
11. Dean (Student Affairs) – Dr. Ebby Darney
12. CoE – Dr. Sadashivappa G.
13. Dean Research (R&D) – Dr. T. C. Manjunath
14. Dept. Staff Members – Teaching & Non-teaching
15. Family members and relatives
16. Friends and the Almighty

We owe a particular debt to the laboratory staff for access to the electronics bench, the measurement equipment and the department network during the hardware trials, and to our classmates who acted as test drivers and whose unreadable plates, mistyped bookings and determined attempts to defeat the gate produced most of the defect log reported in Chapter 12.

TABLE OF CONTENTS

Title Sheet 1
Certificate 2
Declaration 3
Acknowledgement 4
Table of Contents 5
List of Figures and Tables 6
Nomenclature and Acronyms 8
Abstract 10

1. Introduction and Organization of the Project Report 11
2. Objective / Aim / Goal of the Project Work 13
3. Motivation and Problem Statement Definition 16
4. Literature Survey and Gaps 19
5. Design Concepts Used in the Project 25
6. Tools Used in the Project (Hardware and Software) 29
7. Proposed Algorithm, Flow Chart and Data-Flow Diagrams 32
8. Working Principle of the Project 37
9. Proposed Block Diagram and Circuit Diagram 40
10. Proposed Methodology 44
11. Overall Working of the Project 47
12. Results (Simulation / Experimental) and Justifications 54
13. Applications, Advantages and Drawbacks 66
14. Outcome of the Project and Reflection Notes 68
15. Conclusions and Future Works that Could be Done 71
References and Bibliography 74
Appendix 76
PO and PSO Mapping for Project Work 81
Project Cost 82
Papers Published 83
Project Competitions Participated and Won 84
KSCST or Other Funding Applied and Grants Received 85
Patents Filed and Published 86

LIST OF FIGURES

Fig. 1 : System context diagram 25
Fig. 2 : Capture and reading stages 1–4 of the recognition pipeline 32
Fig. 3 : Decision path, stages 5–8, with allow / deny / hold outcomes 34
Fig. 4 : Parking bay state machine 35
Fig. 5 : Booking state machine 35
Fig. 6 : Parking session state machine 36
Fig. 7 : Block diagram of the proposed system 40
Fig. 8 : Circuit diagram of one gate node 41
Fig. 9 : Entity–relationship diagram of the five-table schema 45
Fig. 10 : Booking sequence — reservation before arrival 48
Fig. 11 : Entry sequence — authorised entry and refusal paths 49
Fig. 12 : Exit sequence — release and billing 50
Fig. 13 : Deployment topology on the facility LAN 51
Fig. 14 : Firmware lifecycle of a gate node 52
Fig. 15 : Live Deck showing derived availability for six bays 57
Fig. 16 : Pre-booking page with the bay bound before arrival 57
Fig. 17 : Operator console with facility plan, gates and open sessions 58
Fig. 18 : Frame captured by the entry node (640×480 JPEG) 60
Fig. 19 : Plate crop and one binarised variant of the same frame 60
Fig. 20 : Bench rig — entry node, servo barrier and lane 63

LIST OF TABLES

Table 1 : Objectives of the project and how each is evidenced 14
Table 2 : Capabilities deliberately kept out of scope 17
Table 3 : Literature survey — methods, advantages and gaps 23
Table 4 : Design decisions, alternatives and accepted trade-offs 27
Table 5 : Hardware used in the project 29
Table 6 : Software used in the project and its role 30
Table 7 : Pin map of a gate node 42
Table 8 : Wiring table — entry node 42
Table 9 : Wiring table — exit node 42
Table 10 : Barrier control truth table 43
Table 11 : Methodology phases and deliverables 45
Table 12 : WebSocket event catalogue 53
Table 13 : Levels of testing used in the project 54
Table 14 : End-to-end automated checks and their results 55
Table 15 : Manual and interface checks 56
Table 16 : Defect log 59
Table 17 : Two diagnostic records captured on the bench 60
Table 18 : Verified functional results, 31 October 2026 61
Table 19 : Engineering metrics measured from the repository 62
Table 20 : Comparison of AutoPark with related approaches 63
Table 21 : Threats to validity and their mitigation 64
Table 22 : Application areas of the system 66
Table 23 : Objectives achieved and evidence 68
Table 24 : Month-wise work flow of the project 73
Table 25 : REST API reference 78
Table 26 : Bill of materials 79
Table 27 : Bench checklist before every demonstration 79
Table 28 : Troubleshooting guide 80
Table 29 : PO and PSO mapping 81
Table 30 : Project cost estimation 82

NOMENCLATURE AND ACRONYMS

Abbreviations (alphabetical order)

ANPR / ALPR	Automatic Number Plate Recognition / Automatic Licence Plate Recognition
API	Application Programming Interface
CIE	Continuous Internal Evaluation
CORS	Cross-Origin Resource Sharing
CSE (IC)	Computer Science and Engineering (IoT and Cyber Security with Blockchain Technology)
CTC	Connectionist Temporal Classification
DFD	Data-Flow Diagram
ER	Entity–Relationship
GPIO	General-Purpose Input/Output
HIL	Hardware-in-the-Loop
HSRP	High Security Registration Plate
HTTP / HTTPS	Hypertext Transfer Protocol / HTTP Secure
IoT	Internet of Things
JPEG	Joint Photographic Experts Group (image format)
JSON	JavaScript Object Notation
JWT	JSON Web Token
KSCST	Karnataka State Council for Science and Technology
LAN	Local Area Network
LSTM	Long Short-Term Memory
OCR	Optical Character Recognition
ORM	Object–Relational Mapping
PDR	Plate Detection Rate
PO / PSO	Programme Outcome / Programme Specific Outcome
PSRAM	Pseudo-Static Random Access Memory
PWM	Pulse-Width Modulation
REST	Representational State Transfer
SEE	Semester End Examination
SQL / SQLite	Structured Query Language / embedded SQL database engine
UG BE	Under-Graduate Bachelor of Engineering
UI / UX	User Interface / User Experience
UTC	Coordinated Universal Time
VGA	Video Graphics Array, 640×480 pixels
VTU	Visvesvaraya Technological University
WS	WebSocket

YOLO	You Only Look Once (single-stage object-detection family)

Symbols used (alphabetical order)

n	Number of parking bays in the facility
V	Number of preprocessing variants applied to one plate crop ($V = 4$)
F	Number of frames accumulated in one recognition burst ($F \leq 3$)
L	Length of a plate string in characters
σ^2	Variance of the Laplacian, used as the sharpness metric
θ	Servo shaft angle: 0° closed, 90° open
₹	Indian Rupee
∞	Infinity

ABSTRACT

Finding a free parking space in dense commercial and campus environments is slow, wasteful and difficult to monitor. Conventional “smart” parking installations solve the visibility problem by instrumenting every bay with an ultrasonic, magnetic or infrared sensor, which multiplies hardware cost, cabling and maintenance with lot size. This project presents AutoPark, a smart parking system that replaces per-bay sensing with two camera nodes at the entry and exit gates. Each node captures a frame every five seconds, transmits it to a FastAPI backend, and the backend locates the licence plate with a YOLOv11 detector and reads it with EasyOCR using four preprocessing variants and multi-frame voting.

The system supports both guaranteed advance pre-bookings and unreserved walk-in entry: a pre-booked vehicle is routed to its held bay, while a walk-in vehicle is automatically allocated the next available bay, raising the SG90 boom barrier and highlighting the assigned bay on the live dashboard. If the lot is full, or if the evidence for a plate is ambiguous, the barrier remains closed. Exit is handled symmetrically: the plate is matched against active sessions, the bay is released, and an itemised receipt is produced. Because occupancy is derived from entry and exit events rather than measured bay by bay, the sensing hardware is constant in the number of bays instead of growing with it.

The implementation comprises a six-bay facility, a real-time Next.js dashboard, a role-aware operator console, a monochrome operations design system, and a hardware-in-the-loop simulator that exercises the same endpoints as the physical cameras. Verification is reported as it stands rather than as it was hoped to stand: twelve of fifteen end-to-end checks pass, eleven functional behaviours were confirmed directly through the HTTP interface — bay binding, walk-in allocation, duplicate suppression, low-confidence refusal, full-lot refusal, exit release and both tariff boundaries — and the complete fifty-seven-test suite is red only for two identified reasons, a missing OCR dependency and six assertions that predate the walk-in policy. Six defects found during validation were fixed and six remain open, all disclosed. Recognition accuracy and end-to-end latency are not claimed as measured results in this report: the measurement protocol is defined, two representative diagnostic records are reproduced, and the remaining bench tests are listed explicitly as pending. The report concludes that gate-only occupancy inference is workable at the scale of a department lot, and that its costs — dependence on plate-read quality, one recovery path for unreadable exits, and a policy decision about unbooked arrivals — are predictable and manageable.

Keywords / Index terms : *smart parking, automatic number plate recognition, YOLOv11, EasyOCR, edge IoT, ESP32-CAM, boom barrier, FastAPI, decision under uncertainty, real-time systems.*

CHAPTER 1

Introduction and Organization of the Project Report

Parking is one of the few everyday engineering problems that is solved almost everywhere by adding hardware rather than by using information that already exists. A driver who reaches a gate has to be told, by a sensor in every bay or by a person in a cabin, whether a space is free. A facility owner who wants to know how full the lot is must either walk it or instrument it. Both answers cost money in proportion to the number of bays, and both stop working when the lot is extended, because every new space brings its own device, its own cable and its own failure mode.

This project starts from a different observation: a parking bay is free if and only if a vehicle has entered the facility and has not yet left it. That sentence is the whole design. If the entry and the exit of a lot are observed reliably, the state of every bay inside the lot can be derived from those two observations, and no device is needed inside the lot at all. AutoPark is an implementation of that idea. Two ESP32-CAM nodes watch the entry and the exit gate. Each posts a JPEG frame every five seconds to a FastAPI backend, which locates the number plate with a YOLOv11 detector, reads it with EasyOCR, matches it against bookings or active sessions, opens an SG90 boom barrier when the evidence is strong enough, and writes every decision — including every refusal — to an audit table. The occupancy of the six bays is then a query over the session table, not a measurement.

The project is deliberately a complete system rather than a recognition demonstration. A driver can register, pre-book a specific bay, arrive, see the assigned bay on a pass screen, leave, and receive an itemised receipt. An operator can watch the facility plan, inspect why a particular vehicle was refused, release a bay that has become stuck because its exit plate was unreadable, and reset the facility between demonstrations. A simulator reproduces the camera traffic through the same HTTP endpoints, so the whole system can be demonstrated and tested without the physical rig. The interesting engineering is therefore not the detector or the OCR engine, both of which are standard components, but the decision policy that sits between them and the barrier: when the evidence is insufficient, the system must refuse, because one wrong admission corrupts the occupancy state, the bill and the audit trail at the same time.

The work was carried out over the sixth and seventh semesters as a Phase–1 / Phase–2 pair. Phase–1 established the problem, the survey, the architecture and the recognition pipeline. Phase–2 completed the authorisation policy, walk-in allocation, the operator console, the billing path, the automated verification suite and the hardware rig, and produced the measurements and the honest list of what remains unmeasured that appear in Chapter 12.

1.1 Organization of the project report

The report is organised in the sequence prescribed for the seventh-semester Phase–2 submission. It begins with the front matter — certificate, declaration, acknowledgement, contents, lists of figures and tables, nomenclature and abstract — and then proceeds through fifteen chapters and the required back matter.

Chapter 1 introduces the problem setting, the proposition and the scope of the work, and describes how the report is organized. **Chapter 2** states the objectives in a form that can be verified, together with the aim, the goal and the sequence of steps by which the goal is reached. **Chapter 3** records the motivation for undertaking the work and defines the problem statement precisely, including what has been deliberately excluded. **Chapter 4** surveys the three bodies of literature on which the project

builds — occupancy sensing, automatic number plate recognition, and low-cost edge vision with actuation — and isolates the research gaps in a comparative table. **Chapter 5** explains the design concepts: the layered edge–gateway architecture, the derived-state model of occupancy, the refusal-by-default authorisation policy, OCR-aware string matching and fail-closed actuation.

Chapter 6 lists the hardware and software tools actually used, with the version of each component as it appears in the repository. **Chapter 7** presents the algorithms, flow charts, data-flow diagrams and state machines that implement the design. **Chapter 8** explains the working principle of the system as a single narrative, from the capture of a frame to the movement of the barrier. **Chapter 9** gives the block diagram, the circuit diagram of a gate node, the pin map, the wiring tables and the power topology. **Chapter 10** describes the methodology: the phases, the development method, the verification strategy and the measurement protocol. **Chapter 11** walks through the overall working of the system for the four flows that matter — pre-booked entry, walk-in entry, exit and operator exception handling — with sequence diagrams.

Chapter 12 reports the results: the automated test suite, the interface checks, the defect log, two diagnostic records captured on the bench, the engineering metrics measured from the repository, the analytical scaling argument, a comparison with related approaches, and an explicit list of measurements that have not yet been made. **Chapter 13** discusses applications, advantages and limitations. **Chapter 14** states the outcome of the project against each objective and records the reflection notes of the team. **Chapter 15** concludes, sets out the scope for future work, and gives the month-wise workflow of the project.

The back matter contains the references in IEEE format, an appendix with code excerpts, the API reference, the bill of materials, the assembly and bench procedures, the PO/PSO mapping, the project cost estimate, and the required declarations on papers, competitions, funding and patents. Every reference cited in the body appears in the reference list, and every reference in the list is cited in the body.

CHAPTER 2

Objective / Aim / Goal of the Project Work

The objectives below are written so that each one can be checked against something observable — a test result, an API response, a database row or an inspection of the code — rather than against an intention. Each objective states what is to be achieved and, in one sentence, how the achievement is evidenced. The evidence is repeated in Table 1 and again, with outcomes, in Table 23 of Chapter 14.

2.1 Objectives

Objective 1 — Detect and read vehicle plates from gate frames. Build a recognition pipeline that accepts a JPEG frame from a low-cost camera, locates the plate region, reads the characters and returns a ranked list of candidates with a confidence for each, or an explicit failure stage when no reading is possible. Evidence: the pipeline runs end to end and exposes per-stage diagnostics through `/api/anpr/diagnostics`.

Objective 2 — Authorise pre-booked and walk-in vehicles. Admit a vehicle whose plate matches exactly one pending booking, and admit an unreserved vehicle by allocating the next free bay automatically, while refusing when the plate is unknown and walk-in admission is disabled. Evidence: simulated entry tests for booked, walk-in and unknown plates.

Objective 3 — Honour the bay that the driver booked. When a booking names a specific bay, the entry allocator must bind that bay if it is still free, and must not silently substitute a different one. Evidence: an executed test in which bay B3 is requested at booking and B3 is the bay allocated at entry.

Objective 4 — Derive occupancy from gate events. The status of a bay must change only as a consequence of a session being created or closed, never as a result of a sensor reading inside the lot. Evidence: layout and summary responses remain consistent with the session table after every entry and exit.

Objective 5 — Control a physical barrier automatically. Drive an SG90 boom barrier from 0° to 90° and back after five seconds, but only when the backend returns an explicit authorisation for that gate. Evidence: the response contract and the firmware truth table; the bench actuation test is listed as pending in Chapter 12 rather than claimed.

Objective 6 — Provide real-time visibility. Broadcast every state transition over a single WebSocket channel so that the driver dashboard, the pass screen and the operator console update without a page refresh, and recover correctly after a dropped connection. Evidence: events observed simultaneously on three clients.

Objective 7 — Produce auditable billing. Compute the charge at exit from the session duration using a single server-side tariff (₹30 for the first hour and ₹20 for every additional started hour), and present it as an itemised receipt that cannot be edited by the client. Evidence: tests at the sixty-minute and sixty-one-minute boundaries.

Objective 8 — Allow demonstration without hardware. Provide simulator endpoints that create real sessions, real bills and real audit rows through the same code paths as the cameras, so that the system can be demonstrated and regression-tested on a laptop. Evidence: the entire automated suite runs through the simulator.

#	Objective	How it is met in the build	How it is evidenced
O1	Detect and read plates from gate frames	YOLOv11 plate detector, four preprocessing variants, three-frame burst voting	Working pipeline; per-stage diagnostics API
O2	Authorise pre-booked and walk-in vehicles	Unique match against pending bookings, or dynamic allocation of the first free bay	Simulated walk-in and booking tests; full-lot refusal
O3	Honour the bay chosen at booking	Booking binds a slot identifier; the allocator reads it before falling back	Executed test: B3 requested, B3 allocated
O4	Derive occupancy from gate events	Bay state machine driven only by session creation and closure	Layout and summary consistent after entry and exit
O5	Control a physical barrier	SG90 servo, 0°/90°, five-second hold, gated on the backend flag	Contract and truth table; bench test pending
O6	Provide real-time visibility	Single WebSocket channel with snapshot-on-reconnect	Live updates on three clients without refresh
O7	Produce auditable billing	Duration computed at exit; ₹30 first hour, +₹20 per started hour	Receipts page and exit response verified
O8	Demonstrate without hardware	Hardware-in-the-loop simulator calling the same endpoints	Authorised and unauthorised simulations, reset flow

Table 1 : Objectives of the project and how each is evidenced

2.2 Aim of the project

To design, implement and validate a smart parking system in which the occupancy of a multi-bay facility is inferred from events observed at the entry and exit gates rather than measured by a sensor in every bay, while preserving accurate availability information, controlled vehicle access and an auditable billing record.

2.3 Goal of the project

The goal is a working six-bay prototype, deployed on a facility LAN, in which a driver can reserve a bay in advance or arrive without one, a camera at each gate reads the plate, the server decides whether the barrier should open, the bay is bound and released correctly, a charge is computed at exit, and every decision — admit or refuse — can be reconstructed afterwards from stored evidence. The success criterion is not recognition accuracy on a benchmark; it is that the substitution of two gate cameras for six bay sensors holds together as a system, with the costs of the substitution stated plainly.

2.4 How the goal is achieved

1. **Define the problem and the boundary.** Establish what is in scope (one lot, two gates, pre-booking, walk-in entry, billing, monitoring) and what is not (payment processing, multi-lot management, per-bay sensors), and state the reasons.
2. **Survey the literature and isolate the gap.** Compare per-bay sensing, overhead vision and gate-based recognition; examine ANPR pipelines and their evaluation methods; conclude that gate-only occupancy inference is practised commercially but rarely analysed as an engineering design.
3. **Design the architecture and the decision policy.** Place all state on the server, reduce the edge device to a sensor and an actuator, define the occupancy invariant, and specify a refusal-by-default policy with numeric thresholds for confidence, ambiguity and duplicate suppression.

4. **Implement the pipeline and the service.** Build the recognition pipeline, the allocator, the billing engine, the REST and WebSocket surface and the web client; integrate the ESP32-CAM firmware and the SG90 barrier.
5. **Verify with an automated suite and a bench rig.** Exercise booking, entry, duplicate suppression, exit, cancellation, refusal, ambiguity and both tariff boundaries against an isolated database; run the interface checks; and list the hardware checks that remain.
6. **Report honestly and hand over.** Document the defects found and fixed, the metrics that were measured and the ones that were not, the limitations and the roadmap, so that the next batch can continue the work from a truthful baseline.

CHAPTER 3

Motivation and Problem Statement Definition

3.1 Motivation

The motivation for the project came from watching the same pattern repeat in the streets around the college and in the commercial areas of Bengaluru. Cars slow down at every side road looking for a space, queue behind a barrier while an attendant writes a number on a slip, and then circle again on the way out because nobody recorded when they came in. The inefficiency is visible, the data to fix it exists, and the hardware normally proposed to capture that data is disproportionate to the size of the lot.

Driver experience. Arriving without knowing whether a bay is free causes circling, delay and congestion. The classical study of cruising for parking estimated that a substantial share of traffic in congested commercial districts — of the order of thirty per cent in the districts studied — consists of vehicles searching for a space [2]. A driver who knows before turning in that a bay is held for the vehicle removes most of that search, and a driver who arrives without a booking benefits from being told immediately which bay has been assigned.

Operator economics. In a sensor-per-bay design, every additional bay adds a device, a power drop, a mounting point, a cable run and one more thing that can fail. The cost of the sensing layer therefore grows linearly with capacity, and retrofitting an existing unmonitored lot is invasive because cabling has to reach every space. A gate-only design adds no marginal sensing hardware per bay: the second camera is the last camera, whatever the size of the lot. For a department lot of six bays the difference is modest in absolute terms, but the shape of the cost curve is what matters, and it is the property the project set out to demonstrate.

Accountability. A plate-indexed audit trail links every entry, exit and charge to a vehicle. A manned gate produces a paper slip that is easy to lose and easy to dispute; a gate that records the plate, the confidence, the failure stage and the decision produces evidence that can be examined after the fact. This matters not only for revenue but for the safety of the decision: when a vehicle is refused, the operator should be able to see whether the camera was blurred, whether the plate was read but unmatched, or whether two bookings were too similar to separate.

Practicality for a campus. A six-bay pilot is representative of a department lot, is buildable with hobby-grade hardware within an academic budget, and exercises every part of a real system: authorisation, actuation, billing and monitoring. A larger pilot would have demonstrated the same properties at greater cost and with more opportunity for the demonstration to fail for reasons unrelated to the design.

Academic motivation. Most student projects in this area stop at a detector that draws a box around a plate. The part that is genuinely difficult — deciding what to do when the detector is not sure — is usually skipped. The team was interested in that part: in treating recognition as evidence rather than as an answer, and in building a system whose failure behaviour is as defensible as its success behaviour.

3.2 Problem statement

Small and medium parking facilities cannot afford to instrument every bay, yet drivers and operators need live availability, controlled access and an auditable record of who parked where and for how long. The two remedies available today are both unsatisfactory at this scale. Per-bay sensors give accurate occupancy but require one device, one power drop and one mounting point per bay, so capital and maintenance cost grow linearly with capacity and retrofitting an existing lot is disruptive. Fully manual gates are cheap to install but cannot provide live availability, an audit trail or automated billing, and they depend on an attendant being present.

There is therefore a gap for a system that provides live bay availability, automated access control and auditable billing, using sensing hardware whose cost does not scale with the number of bays, and whose behaviour when a plate cannot be read is defined rather than accidental.

Formally, the system to be built must satisfy the following. Given a facility of n bays served by exactly two camera nodes, one at the entry gate and one at the exit gate, the system must maintain a bay-state function $S(t)$ that is correct with respect to the sequence of admitted entries and completed exits; must admit a vehicle only when the plate evidence resolves to exactly one authorised vehicle or when an unreserved bay can be allocated; must refuse, with a recorded reason, in every other case; must compute a charge from the interval between admission and release; and must expose the resulting state to drivers and operators within two seconds of the change. The sensing cost of the system must be independent of n.

3.3 Scope and boundary

Two capabilities that a commercial product would require are deliberately excluded, and the reason for each exclusion is stated so that the boundary is a design decision rather than an omission. Unreserved walk-in entry, by contrast, is fully integrated: it is part of the problem rather than an extension, because a real lot cannot operate on reservations alone.

Excluded	Reason for exclusion
Online payment	Adds a payment-gateway integration and a compliance surface that has no bearing on the central hypothesis. The bill is computed, itemised and stored; no money is moved.
Multi-level and multi-lot management	The architecture generalises, but a second lot multiplies hardware and does not test the claim. The schema currently has no lot identifier; adding one is listed as future work.
Native mobile application	The web client is responsive and usable on a phone in a browser; a native client is a packaging exercise rather than an engineering contribution.
Per-bay sensors	Excluded by design — the substitution of gate observation for per-bay measurement is the hypothesis under test.
Vehicle make, colour and damage capture	Not required for authorisation or billing, and would add a second model and a privacy surface without changing the occupancy argument.

Table 2 : Capabilities deliberately kept out of scope

3.4 Terms used in this report

Bay. A single parking space, identified by a slot number such as A2. The pilot has six: A1–A3 in zone A and B1–B3 in zone B.

Booking (pre-registration). A reservation made before arrival. It binds a specific bay and expires at a stated time. A booking holds the bay: the bay is not offered to anyone else while the booking is

pending.

Session. The record of a vehicle being inside the facility. It is created at entry and closed at exit, and it carries the entry time, the exit time, the bay, the plate and the evidence behind the admission decision.

Occupancy. In this report, a derived property: a bay is occupied if and only if an active session references it. It is never read from a sensor inside the lot.

Candidate. One plate string produced by the OCR stage, together with a confidence and the preprocessing variant that produced it.

Refusal. Any decision that leaves the barrier closed. Every refusal carries a machine-readable status such as `no_plate`, `unauthorized`, `ambiguous`, `already_allocated` or `parking_full`.

CHAPTER 4

Literature Survey and Gaps

This chapter positions AutoPark against the bodies of work it builds on: how parking occupancy is sensed, how number plates are recognised, how low-cost edge devices are used for vision and actuation, how real-time monitoring interfaces are built, and how access control and audit are handled in a web application. The aim is not to catalogue the literature but to isolate the gap that the project addresses, and to show that each design choice has a justification in prior work.

4.1 Occupancy sensing in smart parking

Lin, Rivano and Le Mouël classify smart parking systems by the sensing layer used to detect the presence of a vehicle [1]. The dominant approach is a sensor per bay: an ultrasonic, magnetometer, infrared or inductive-loop device mounted in or beside each space, reporting occupancy to a gateway. These systems are accurate because they measure the state of each bay directly, and commercial deployments are common in multi-storey facilities. Rajabioun and Ioannou take the complementary approach of predicting availability from historical and spatio-temporal data rather than measuring it [25], which is useful for advising a driver before the journey but does not replace access control.

The economic structure of the per-bay approach is the problem. Every bay requires one device, one power drop, one mounting point and one more thing that can fail. For a facility with n bays, the sensing layer grows as $O(n)$ in capital cost, installation time and maintenance. Retrofitting an existing unmonitored lot is invasive, because cabling must reach every space.

A second family of designs uses vision from a vantage point: a camera overlooking a row or a level, with spaces delineated in the image and a classifier deciding occupancy per region. This amortises hardware over many bays, but each camera must see the spaces it serves, so coverage still scales with the geometry of the lot, and occlusion by large vehicles degrades accuracy. Camera-based occupancy systems also carry privacy and storage costs because they record the interior of the facility continuously.

Gate-based recognition, the approach taken here, sits between these families. The camera observes only the entrance and the exit, so two devices serve an arbitrary number of bays. Occupancy is not measured at all; it is inferred from the sequence of entries and exits. This is the same principle used by ticket-based and barrier-controlled commercial car parks, where the ticket count, rather than a sensor, is the occupancy record. The contribution of this project is to replace the ticket with an automatically recognised plate, which makes the record auditable without a human at the gate.

4.2 Automatic number plate recognition

Automatic number plate recognition has a long literature; Du et al. survey the classical pipeline of plate localisation, character segmentation and character recognition [3]. Classical systems rely on edge statistics, morphology and colour cues to find the plate, then on template matching or a small classifier per character. They work well under controlled illumination and fixed camera geometry, and degrade quickly when the plate is small, skewed or unevenly lit.

Two developments changed the practical design. First, single-stage object detectors such as YOLO reframed localisation as a regression problem solved in one network pass, giving real-time detection at modest cost [4]. Multi-scale variants such as SSD [21] and later members of the YOLO family [26]

improved the detection of small objects, which is exactly the regime of a plate seen from a gate camera. Modern plate-specific models, including YOLO11 [5], are accurate enough to replace the hand-crafted localisation stage entirely; Laroca et al. demonstrated a complete real-time ALPR pipeline built on a YOLO detector [19], and Silva and Jung extended the idea to unconstrained scenarios with a segmentation-free recognition network [20].

Second, deep-learning OCR engines removed the need for explicit character segmentation. Sequence models trained with connectionist temporal classification [24] over recurrent networks [23] made it possible to decode a whole text line without cutting it into characters; CRNN [28] and the EAST detector [27] are the reference designs for scene text. EasyOCR [6] packages these ideas behind a simple interface and returns a confidence per token, which suits an unconstrained scene such as a plate photographed by a low-cost camera. Underlying all of these are convolutional feature extractors of the VGG [22] and residual families, and the deep-learning frameworks and annotated datasets that made them reproducible [17], [18].

A recognition pipeline assembled from these parts is accurate but not certain. That distinction matters for access control: a character error rate of a few per cent is acceptable for a toll audit, but a single wrong admit corrupts the occupancy state of a lot. The literature on ANPR evaluation therefore reports precision and recall per condition rather than a single accuracy figure, because performance is strongly condition-dependent [3].

The design consequence adopted in this project is to treat recognition as evidence rather than as an answer. Multiple preprocessing variants of the same crop are read independently, the readings are scored for confidence and agreement, several frames are accumulated, and the resulting candidates are compared against known registrations using a string-distance metric [8] rather than a single exact comparison. When the evidence does not resolve to one registration, the system refuses.

4.3 Low-cost edge vision and actuation

The ESP32 family provides a microcontroller with integrated Wi-Fi at a price point that makes a camera node disposable [9]. The ESP32-CAM variant adds an OV2640 sensor and PSRAM, enough to capture and upload a VGA JPEG without a companion processor. This class of device is widely used in IoT prototypes because it removes the need for a single-board computer at the edge: capture, transport and actuation can all run on one microcontroller.

Actuation at the gate is normally a barrier arm driven by a DC motor, a solenoid or a hobby servo. A servo is attractive for a prototype because it is position-controlled: the controller commands an angle rather than a direction, and no limit switches are needed to stop the arm. The SG90 class of micro-servo is cheap and adequate for a lightweight arm, at the cost of torque and of having no position feedback to the controller [10].

The architectural lesson taken from this literature is that the edge device should be a sensor and an actuator, not a decision maker. Deployments that place authorisation logic on the camera node struggle to keep state consistent across nodes, because two nodes can disagree about the state of the lot and there is no shared transaction. AutoPark therefore keeps all decisions on the server and reduces the firmware to capture, upload, and move the arm when told.

4.4 Real-time monitoring interfaces

Operator dashboards need to reflect state changes within a second or two. The classical solution is client polling of a REST endpoint [15], which is simple but generates redundant requests and introduces a polling interval of latency. Server-sent events and WebSockets invert the direction: the server pushes each transition to connected clients. For a small number of screens and infrequent events, a single WebSocket channel is both simpler to operate than a message broker and more responsive than polling.

The relevant design detail, often omitted in student projects, is recovery. A WebSocket stream does not replay missed messages, so a client that reconnects after a network drop must re-read a snapshot before resuming the stream. This project implements that snapshot-then-stream pattern, and it is the reason the dashboard can be trusted after a laptop sleeps or a Wi-Fi handover.

4.5 Access control, identity and audit

The web tier of a parking product is a conventional authenticated application. JSON Web Tokens [14] with a server-side signature provide stateless session identity, and password storage follows the OWASP recommendation of a slow, salted hash [16]. The framework choices — FastAPI for a typed HTTP API [11], SQLAlchemy for the persistence layer [13], OpenCV for image processing [7] and Next.js for the client [12] — are unremarkable; they are selected because they minimise the code needed for the parts of the system that are not the contribution.

The audit requirement is more interesting. A gate system must be able to answer, after the fact, why a particular vehicle was admitted or refused. That requires recording the evidence behind each decision, not only its outcome. This is why the schema contains a detection-event table separate from the session table, and why the recognition pipeline exposes per-stage diagnostics rather than a single success flag.

4.6 Literature survey table

Table 3 summarises the works that shaped the design. The final two columns record what each work contributes and where it stops, because the gaps column is what the project is built on.

Ref.	Authors et al.	Title / focus	Method and contribution	Advantages	Disadvantages / gaps
[1]	Lin, Rivano, Le Mouël, 2017	A survey of smart parking solutions	Taxonomy of sensing, communication and application layers	Comprehensive map of the field; cost structure made explicit	No gate-only occupancy model; failure behaviour not analysed
[2]	Shoup, 2006	Cruising for parking	Empirical study of search traffic in commercial districts	Quantifies the cost of not knowing availability	Policy study, not a system design
[3]	Du, Ibrahim, Shehata, Badawy, 2013	ALPR: a state-of-the-art review	Classical localisation, segmentation and recognition pipeline	Clear account of condition dependence	Pre-deep-learning; no decision policy
[4]	Redmon et al., 2016	You only look once	Single-stage detection as regression	Real-time detection at modest cost	Weak on small objects; no plate specialisation

Ref.	Authors et al.	Title / focus	Method and contribution	Advantages	Disadvantages / gaps
[5]	Ultralytics, 2024	YOLO11 documentation and models	Current detector family with training recipes	Strong accuracy / speed trade-off; easy fine-tuning	Requires a labelled plate dataset
[6]	Jaided AI	EasyOCR	Scene-text detection and recognition with per-token confidence	No character segmentation; 80+ languages	Weak on low-resolution, blurred crops
[7]	Bradski, 2000	The OpenCV library	Image processing primitives	Fast, portable; thresholding and filtering	Library, not a method
[8]	Levenshtein, 1966	Binary codes capable of correcting deletions and insertions	Edit-distance metric	Principled tolerance of single-character errors	No notion of visual confusion or confidence
[9]	Espressif, 2023	ESP32 series datasheet	Microcontroller with integrated Wi-Fi	Cheap, capable edge node	Limited RAM; no secure element by default
[10]	TowerPro	SG90 micro-servo datasheet	Position-controlled hobby servo	Angle commanded directly; no limit switches	No position feedback; limited torque
[11]	Ramírez	FastAPI documentation	Typed Python HTTP framework	Automatic validation and OpenAPI docs	Single-process concurrency model
[12]	Vercel	Next.js documentation	React framework with App Router	Server / client boundaries; fast iteration	Client state must be reconciled with the server
[13]	SQLAlchemy	SQLAlchemy 2.0 documentation	ORM and query layer	Portable models; transactions	SQLite write concurrency limits
[14]	Jones, Bradley, Sakimura, 2015	JSON Web Token (RFC 7519)	Signed stateless identity token	Simple for browsers and devices	No revocation without extra infrastructure
[15]	Fielding, 2000	Architectural styles for network-based software	REST architectural constraints	Resource-oriented, cacheable API design	Polling-based updates add latency
[16]	OWASP	Password storage cheat sheet	Slow salted hashing guidance	Defensible credential storage	Guidance only; no rate limiting
[17]	Deng et al., 2009	ImageNet	Large-scale labelled image database	Enabled transfer learning for detection	General objects, not plates
[18]	Paszke et al., 2019	PyTorch	Deep-learning framework	Reproducible training and inference	Heavy dependency for a small deployment
[19]	Laroca et al., 2018	Real-time ALPR based on YOLO	End-to-end ALPR with a YOLO detector	Fast enough for gate use; high recall	Fixed camera geometry; no authorisation policy
[20]	Silva, Jung, 2018	Plate detection in unconstrained scenarios	Segmentation-free recognition network	Robust to viewpoint and lighting	Computationally heavy for an edge server
[21]	Liu et al., 2016	SSD: single shot multibox detector	Multi-scale single-stage detection	Better small-object recall than early YOLO	Still needs plate-specific training

Ref.	Authors et al.	Title / focus	Method and contribution	Advantages	Disadvantages / gaps
[22]	Simonyan, Zisserman, 2015	Very deep convolutional networks	Deep CNN feature extractor	Strong transfer features	Large parameter count
[23]	Hochreiter, Schmidhuber, 1997	Long short-term memory	Recurrent sequence model	Handles variable-length text lines	Needs substantial training data
[24]	Graves et al., 2006	Connectionist temporal classification	Alignment-free sequence decoding	Removes explicit character segmentation	Assumes monotonic alignment
[25]	Rajabioun, Ioannou, 2015	Parking availability prediction	Spatio-temporal multivariate model	Useful pre-trip advice	Predicts, does not authorise or bill
[26]	Bochkovskiy et al., 2020	YOLOv4	Improved single-stage detector	Strong accuracy at real-time speed	Heavier than needed for a six-bay lot
[27]	Zhou et al., 2017	EAST scene-text detector	Direct regression of text regions	Fast, rotation-aware	Scene text, not plate-specific
[28]	Shi, Bai, Yao, 2017	End-to-end scene-text recognition	CRNN with CTC decoding	Whole-line recognition	Recognition only; localisation separate

Table 3 : Literature survey — methods, advantages and gaps

4.7 Research gaps observed

Gap 1 — sensing cost scales with the number of bays. Occupancy sensing is the cost driver in smart parking, and virtually every published design instruments each bay or each zone [1]. Gate-only occupancy inference is used in practice by ticket-based operators, but it is rarely treated as an academic design and its failure modes are rarely analysed. No surveyed work quantifies what happens to availability accuracy when the only observation is at the boundary of the lot.

Gap 2 — recognition accuracy is reported, decision error is not. ANPR papers usually report a single accuracy or a precision/recall pair for the recognition stage [3], [19], [20]. The property that matters for access control is the decision error rate — false admits and false rejects — under a refusal policy. A system that refuses when uncertain has a different error profile from one that always returns its best guess, and the two cannot be compared on recognition accuracy alone.

Gap 3 — integration and audit stop short. Prototypes typically demonstrate detection, or allocation, or a dashboard, but not recognition, authorisation, actuation, billing and monitoring as one auditable pipeline. The evidence behind a decision is usually discarded, so the system cannot explain a refusal after the fact.

Gap 4 — low-cost edge hardware is used as a camera, not as a system component. Many IoT parking prototypes put decision logic on the edge node, which makes state consistency across nodes difficult. The alternative — a deliberately stupid edge and a single authoritative server — is well established in practice but under-reported in student work.

4.8 Conclusive remarks of the survey

The survey supports three conclusions that shaped the design. First, the sensing architecture, not the recognition algorithm, is where the cost of a smart parking system is decided, and a gate-only architecture is a legitimate and under-studied alternative for small lots. Second, plate recognition must

be treated as evidence with a confidence and a failure stage, and the decision to admit must be separated from the recognition itself so that ambiguity can be refused rather than guessed. Third, the value of the system depends on the integration: a plate read that is not linked to a booking, a bay, a bill and an audit record is a demonstration, not a parking system.

AutoPark addresses the three gaps together: a two-camera architecture whose sensing cost is constant in the number of bays, a refusal-by-default authorisation policy with OCR-aware matching, and an end-to-end implementation whose decisions are recorded with evidence. The next chapter describes the design concepts that implement these conclusions.

CHAPTER 5

Design Concepts Used in the Project

This chapter explains the design concepts that hold the system together. They are presented as concepts rather than as components because each one is a rule that constrains many parts of the implementation: the architecture rule decides where state lives, the occupancy rule decides what a bay status means, the refusal rule decides what happens when the evidence is weak, and so on. Together they form the conceptual framework within which the algorithms of Chapter 7 and the implementation of Chapters 8–11 are written.

5.1 Edge–gateway architecture with a single source of truth

AutoPark is an edge–gateway, event-driven architecture with a thin browser client. The edge tier consists of two ESP32-CAM nodes that do exactly two things: capture a JPEG every five seconds and actuate a servo when told to. They hold no state about the lot. The application tier is a FastAPI service that owns all decisions — identity, plate matching, bay allocation, billing and audit — and is the single source of truth. The presentation tier is a Next.js application that renders state and reacts to WebSocket events; it never computes occupancy. The data tier is SQLite accessed through SQLAlchemy, with five tables.

The important architectural consequence is that cameras are sensors, not controllers. If a camera is replaced, the lot state is unaffected. If the backend is down, the barriers fail closed, because the firmware opens the arm only on an explicit boolean in the response. If the browser is closed, nothing about the lot changes. Every one of those properties follows from putting the state in one place.

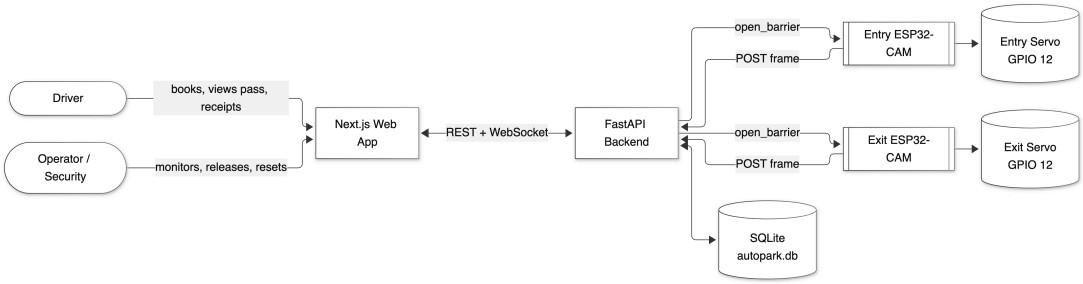


Fig. 1 : System context diagram

5.2 Occupancy as a derived state

A bay is occupied if and only if an active session references it. This is the central design concept of the project, and it is enforced structurally rather than by convention. A bay becomes occupied when the allocator creates a session inside a transaction that also changes the bay status; it becomes free when the exit path closes that session inside a transaction that also restores the bay status. There is no other writer. The dashboard, the summary endpoint and the operator plan all read the same derived value, so they cannot disagree with each other or with the audit trail.

The consequence is a different kind of accuracy claim from a per-bay sensor. A sensor measures the bay continuously and is right about the bay even when it is wrong about the vehicle. A derived state is exactly right about the vehicle and can be wrong about the bay if an event is missed — specifically, if a vehicle leaves and its plate is never read. The design accepts that failure mode and provides a

documented manual release for it, rather than pretending that the derived state is as robust as a measurement. The trade is stated in Table 20 of Chapter 12 and in the limitations of Chapter 13.

5.3 Refusal by default

Authorisation is designed around a simple asymmetry: a false reject costs one retry, while a false admit creates a wrong bay, a wrong bill and a corrupted occupancy map. The system therefore refuses whenever the evidence is insufficient, and it defines insufficiency numerically rather than intuitively. Four independent gates decide admission:

1. the candidate confidence must be at least 0.75;
2. the plate must contain at least four alphanumeric characters;
3. the reading must match a pending booking (or a free bay must exist for walk-in entry);
4. the best match must be separated from the second-best by at least 0.10 in score.

Because the gates are independent, a failure of one does not silently weaken the others. Ambiguity in particular is treated as a first-class outcome: when two bookings are plausible, the system does not pick the higher score, it stops and raises an operator alert. This inverts the usual "best effort admit" behaviour of demonstration ANPR systems, and it is the property that makes the system defensible in front of an examiner.

5.4 OCR-aware matching

A plate read by a cheap camera is not a string, it is a string plus a set of visual confusions. The letter O and the digit 0 are the same shape at low resolution; so are I and 1, S and 5, B and 8, Z and 2, G and 6. The matcher therefore canonicalises both the candidate and the registered plate through a confusion map before comparing them, then computes a Levenshtein distance on the residual strings, then discounts the resulting similarity by the OCR confidence. The formula is:

$$adjusted = similarity \times (0.80 + 0.20 \times c_{cand})$$

with a minimum adjusted score of 0.72 for a match to be considered at all. The confidence factor prevents a confident wrong read from being discounted into a match, and the floor of 0.80 prevents a marginal read from outranking a strong one. The worked examples in Chapter 7 show both a tolerated single-character error and a refused ambiguity.

5.5 Fail-closed actuation

The barrier has two states and one rule: it opens only when the server says so. The firmware parses the response, reads a boolean, and calls the open routine only if that boolean is true; every other outcome — timeout, malformed JSON, HTTP error, missing field — leaves the arm at zero degrees. The servo is open-loop, so the system never claims to know where the arm is; the user interface reports the commanded state and says so. This is a deliberate degradation design: the failure of any component between the camera and the server results in a closed barrier, which is the safe direction for an access-control device.

5.6 Event-driven presentation

Every state transition is published on a single WebSocket channel as a typed event. Clients subscribe once, patch their local state, and re-read a REST snapshot after any reconnect, because a stream does not replay missed messages. The client is therefore a projection of server state rather than a second source of truth. This is what allows three different screens — the driver dashboard, the pass screen and the operator console — to stay consistent without polling and without duplicating the occupancy logic in the browser.

5.7 Design decisions and trade-offs

Table 4 records the decisions that shaped the system, the alternatives that were considered, and the trade-off that was accepted in each case. The trade-off column is the important one: every decision in this project costs something, and the report states the cost rather than hiding it.

ID	Decision	Alternatives considered	Rationale	Trade-off accepted
D1	Two gate cameras, no bay sensors	Ultrasonic per bay; overhead vision	Constant hardware cost; minimal retrofit	Occupancy depends on plate reads
D2	Hybrid pre-booked and walk-in access	Pre-booked only	Serves advance reservations and spontaneous arrivals	Walk-ins cannot take reserved bays
D3	Backend owns all state	Camera-side decisions	Single source of truth; auditable	Backend is a single point of failure
D4	Refuse on ambiguity	Best-score admit	A false admit corrupts occupancy and revenue	Occasional false rejects
D5	SQLite persistence	PostgreSQL	Zero operations; adequate for one lot	Limited write concurrency
D6	JWT with bcrypt hashing	Session cookies	Stateless API; simple for browser and device	Token revocation is manual
D7	WebSocket push	Polling	Instant UI with less load	Connection lifecycle code
D8	Servo barrier on GPIO 12	Buzzer only	Physical access control that matches real gates	5 V current and timer contention
D9	Billing computed on the server	Client-side estimate	Single authority; receipts cannot be edited	Recomputation on every exit
D10	Simulator endpoints	Mock user interface only	Exercises the real database, allocator and billing path	Simulator can create real sessions

Table 4 : Design decisions, alternatives and accepted trade-offs

5.8 Architectural invariants

The following properties must remain true; violating one is a design regression rather than a bug. They are used as a checklist during review and are restated in the conclusion.

1. No bay is allocated without a plate decision, except through the explicit operator simulator.
2. A bay is occupied only while an active session references it.
3. Occupancy is never computed in the frontend.
4. The barrier opens only on an explicit backend boolean.
5. Every entry, exit and refusal writes a detection-event row with evidence.

6. A booking always holds a concrete bay identifier, and releasing the booking releases the bay.
7. Ambiguity is a refusal, not a coin flip.

5.9 Data model

The schema is deliberately small: five tables, each with one responsibility. *Users* holds credentials and roles; *parking_slots* holds the six bays and their current status; *pre_registrations* holds bookings and binds a bay; *parking_sessions* holds the interval during which a vehicle is inside; and *detection_events* holds the evidence behind every decision. The session table is the authority for occupancy, and the detection-event table is the authority for why a decision was taken. Keeping them separate is what makes the audit trail possible without duplicating state. The entity–relationship diagram is given as Fig. 9 in Chapter 10, where the data design is discussed together with the rest of the methodology.

CHAPTER 6

Tools Used in the Project (Hardware and Software)

This chapter lists the hardware and software actually used to build and run the system, with the version of each component as it appears in the repository, and explains in one or two sentences why that tool was chosen for the problem rather than a more capable alternative. Three constraints governed the selection: the rig had to be affordable within a department project budget, the stack had to run unattended on a single laptop on the facility LAN, and every component had to be something the team could debug at source level when it failed during a demonstration.

6.1 Hardware

The hardware is deliberately minimal: two camera nodes, two servos and the material to mount them. There is no single-board computer at the gate, no sensor in any bay, and no router beyond the department Wi-Fi access point. The camera node is an ESP32-CAM with the AI-Thinker pinout, an OV2640 sensor and 4 MB of PSRAM; the PSRAM matters because it allows a 640×480 JPEG to be captured and buffered without dropping frames. The actuator is an SG90 micro-servo, which is position-controlled and therefore needs no limit switches, at the cost of torque and of having no position feedback. A passive buzzer provides an audible confirmation at the entry gate and can be disabled in firmware.

#	Component	Qty	Specification as used	Role in the system
1	ESP32-CAM (AI-Thinker)	2	ESP32 dual-core 240 MHz, 520 KB SRAM, 4 MB PSRAM, OV2640 camera, 802.11 b/g/n	Captures a VGA JPEG every five seconds and drives the servo on command
2	USB-TTL programmer (3.3 V)	1	FTDI-compatible, 3.3 V logic	Flashing the ESP32-CAM, which has no USB port
3	SG90 micro-servo	2	180° travel, 4.8–6 V, ~1.8 kg·cm stall torque	Boom barrier arm: 0° closed, 90° open, five-second hold
4	5 V 2 A adapter	2	Regulated 5 V, 2 A, common ground with the board	Separate supply for servo and board to avoid brown-out resets
5	Active buzzer	1	5 V active buzzer on GPIO 13	Audible confirmation of an admitted entry; optional
6	Jumper wires and breadboard	—	Male-to-female dupont leads	Prototyping the node without soldering
7	Boom arm material	2	Lightweight cardboard and 3D-printed bracket	Visible barrier arm attached to the servo horn
8	Mounting hardware	—	Cable ties, base plates, clamps	Fixing camera and servo at the gate
9	Host laptop	1	macOS, Python 3.13, Node.js 26, 8 GB RAM	Runs the backend, the database, the models and the web client
10	Facility Wi-Fi / LAN	1	802.11n, 2.4 GHz, 192.168.0.0/24	Carries camera uploads, browser traffic and WebSocket events

Table 5 : Hardware used in the project

Two details of the hardware design are worth recording because they caused most of the bench problems. First, the servo must not be powered from the ESP32-CAM 3.3 V pin; the current draw at stall resets the board, and the symptom looks like a software fault. A separate 5 V rail with a common

ground, and a 470–1000 μF electrolytic capacitor across the rail near the servo, removes the resets. Second, the ESP32-CAM has no USB port, so the programmer must be connected with GPIO 0 held low at boot; the AI-Thinker board silkscreen labels this poorly, and a node that appears dead is usually a board that never entered flash mode.

6.2 Software

The software stack is a conventional Python service with a JavaScript client. FastAPI was chosen because it gives a typed request/response contract and automatic OpenAPI documentation with very little code; SQLAlchemy because the models are simple and the same code would run on PostgreSQL if the deployment grew; SQLite because a single lot with a handful of clients needs no database server; and Next.js because the dashboard needs both server-rendered pages and a long-lived WebSocket connection in the browser. The recognition stack is Ultralytics YOLO11 for localisation, EasyOCR for character recognition, and OpenCV for the quality gate and the preprocessing variants.

Layer	Technology	Version in repository	Role
Edge firmware	Arduino C++ (ESP32 core)	—	Capture, HTTP POST, JSON parse, servo control
Plate detection	YOLOv11 (Ultralytics)	ultralytics (not pinned)	Locates the plate region in the frame
Plate reading	EasyOCR	not listed in requirements	Recognises characters and returns per-token confidence
Image processing	OpenCV (headless)	≥ 4.8.0	Quality metrics, CLAHE, adaptive and Otsu thresholding
Backend framework	FastAPI	≥ 0.100.0	REST and WebSocket API, validation, OpenAPI docs
ASGI server	Uvicorn	≥ 0.22.0	Serves the API, single process
ORM	SQLAlchemy	≥ 2.0.0	Models, queries, transactions
Database	SQLite	Python standard library	Persistence in `backend/autopark.db`
Authentication	PyJWT + bcrypt	≥ 2.8.0, ≥ 4.0.0	HS256 tokens and salted password hashing
Frontend framework	Next.js (App Router)	^14.2.5	Routing and rendering
UI runtime	React	^18.3.1	Components and client state
Language	TypeScript	^5.5.4	Type safety on the client
Styling	Tailwind CSS	^3.4.7	Design tokens and layout utilities
Icons and motion	lucide-react, Framer Motion	^0.417.0, ^11.3.19	Consistent line icons; modal and card transitions
Testing	pytest, httpx	≥ 7.4.0, ≥ 0.24.0	Automated verification against an isolated database
Editor and toolchain	Arduino IDE, VS Code, Git	—	Firmware, source and version control

Table 6 : Software used in the project and its role

6.3 How the tools were used to suit the problem

YOLOv11 and EasyOCR. The detector is used only for localisation, not for recognition. It is loaded from `backend/models/license_plate_best.pt` and the plate pass is invoked with a deliberately

permissive inference threshold of 0.10: a crop that is likely to contain a plate but not certain is still worth passing to the OCR stage, because the OCR confidence and the matching policy provide the real decision. Acceptance at 0.75 is applied to the frame confidence after voting, not before. A general vehicle model (`yolo11n.pt`) is also loaded to classify the vehicle type for the receipt, but the plate model is the one that gates access.

OpenCV. OpenCV is used for three things: the sharpness, brightness and contrast metrics that form the quality gate; the four preprocessing variants (adaptive threshold, inverted adaptive threshold, Otsu and CLAHE); and the debug artefacts that the diagnostics endpoint can return. It is not used for detection or recognition, because classical computer vision is exactly what the detector replaced.

FastAPI and Uvicorn. The service exposes twenty-two REST routes and one WebSocket route. FastAPI's dependency system supplies a database session per request and a current-user dependency for the account routes; its response models keep the contract between the backend and the browser explicit, which is what allowed the client to be written against the schema rather than against prose.

Next.js, React and Tailwind. The client has eight pages, of which five are part of the demonstration flow. The App Router allows the layout, sign band and navigation to be rendered once and the live data to be fetched on the client, where the WebSocket lives. Tailwind carries the design tokens, so the colour of a status is defined once and used consistently in the plan, the table and the legend.

Arduino core for ESP32. The firmware is two sketches of about 350 lines each. They do not contain business logic. The entry sketch captures a frame, posts it to `/api/detection/entry`, parses the JSON response with ArduinoJson, and moves the servo to 90° for five seconds if and only if `open_barrier` is true. The exit sketch is the same without the buzzer. Keeping the firmware this small is a design decision (Section 5.5), not a limitation of the toolchain.

6.4 Development and runtime environment

The backend runs under `/opt/anaconda3/bin/python3` (Python 3.13.9) because the default interpreter on the host machine does not carry SQLAlchemy and the vision dependencies. Both services are started detached by `scripts/host.sh`, which writes logs to `.run/`; the frontend runs on port 3000 and the API on port 8000, and the cameras are pointed at the host laptop's LAN address. There is no cloud dependency, no external database and no TLS terminator: this is a LAN deployment, which is the realistic topology for a gate system. The consequences of that choice — clear-text credentials on the LAN, no rate limiting, unauthenticated detection routes — are listed as limitations in Chapter 13 rather than hidden.

One packaging defect is recorded here because it affects reproducibility: `requirements.txt` lists the web, database and OpenCV dependencies but omits both `easyocr` and `ultralytics`. A clean install therefore produces a backend that starts and serves the dashboard but cannot run live recognition. The defect is carried in the defect log (Table 16) and the fix is part of the future work in Chapter 15.

CHAPTER 7

Proposed Algorithm, Flow Chart and Data-Flow Diagrams

This chapter gives the algorithms that implement the design concepts of Chapter 5, the flow charts that show the order of the stages, the data-flow view of the system, and the state machines that define the legal transitions of a bay, a booking and a session. The algorithms are written in a pseudocode close to the Python implementation so that they can be checked against the code, and each is followed by a short discussion of the decisions it encodes.

7.1 Recognition algorithm

The recognition pipeline turns one JPEG frame into a ranked list of plate candidates, or into an explicit failure with a stage label. It is deliberately not a black box: every stage can fail, and the failure is reported with the reason rather than collapsed into “no plate found”.

```
Algorithm 1  RecognisePlate(frame)
 1  q ← AssessQuality(frame)                        # sharpness, brightness, contrast
 2  if q.grade = invalid then return no_plate(stage=camera_quality)
 3  box, conf ← PlateDetector(frame)                # YOLOv11, inference threshold 0.25
 4  if box is empty then return no_plate(stage=plate_detection)
 5  crop ← frame[box]
 6  variants ← [adaptive, adaptive_inv, otsu, clahe](crop)
 7  candidates ← ∅
 8  for each v in variants do
 9      text, c ← Ocr(v, allowlist="A-Z0-9")
10      if length(text) ≥ 3 and distinct(text) ≥ 2 then
11          candidates ← candidates ∪ {(text, c, v)}
12      if two variants agree on a text of length ≥ 6 with c ≥ 0.75
13          then break                                  # short-circuit, keep the common
case fast
14  for each distinct text t in candidates do
15      votes(t) ← |{v : v produced t}| / V
16      score(t) ← 0.70 · max_conf(t) + 0.30 · votes(t)
17  return candidates ranked by (score, votes)
```

Two details of the algorithm carry most of the weight. The short-circuit at step 12 is what keeps the common case fast: when two variants agree on a long, confident reading, the remaining variants add little and cost an OCR pass each. The scoring at step 16 is what makes agreement count as evidence: a reading seen in three of four variants at 0.78 confidence outranks a reading seen once at 0.80, because the second is more likely to be an artefact of one binarisation. The weights (0.70 confidence, 0.30 agreement) are a design choice, not a fitted parameter, and they are stated as such.

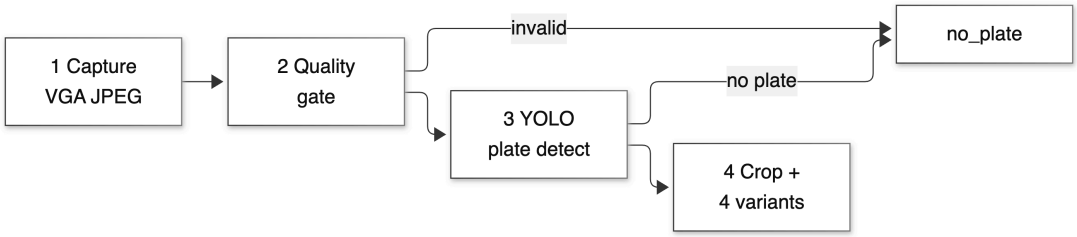


Fig. 2 : Capture and reading stages 1–4 of the recognition pipeline

7.2 Multi-frame burst voting

A single frame is a single sample, and motion blur or JPEG compression can make any one frame unreliable. The pipeline therefore accumulates candidates over a short burst before deciding. The burst window is five seconds, which matches the capture interval of the camera, and at most three frames are merged. A candidate seen in all three frames is promoted over one seen once, even if the latter has a higher single-frame confidence:

$$frame_boost = 0.75 \times max_conf + 0.25 \times frame_ratio$$

where *frame_ratio* is the fraction of frames in the burst that produced the candidate. This is the practical answer to the question "what if one frame is bad?": the system waits for corroboration instead of trusting the first read, and it does so within the same five-second rhythm as the camera, so a moving vehicle is still served promptly.

7.3 Entry allocation algorithm

Allocation is the point at which recognition becomes a physical decision. The algorithm applies the four refusal gates of Section 5.3 in a fixed order, and it holds a single in-process lock while it checks and commits, so two frames arriving together cannot both see the same bay as free.

```
Algorithm 2  AllocateEntry(plate, confidence, candidates)
 1  if confidence < 0.75 then return ignored
 2  if no candidate has ≥ 4 alphanumeric characters then return no_plate
 3  acquire allocation_lock:
 4      if plate matches an ACTIVE session then return already_allocated
 5      if the same canonical plate was seen within 10 s then return already_allocated
 6      match ← unique booking in PENDING_ARRIVAL scoring ≥ 0.72
 7      if match is ambiguous (top two within 0.10) then return ambiguous
 8      if match is none then
 9          if not ALLOW_WALKIN_ENTRY then return unauthorized
10      bay ← match.slot_id if it is still free, else lowest-numbered free bay
11      if no bay then return parking_full
12      bay.status ← OCCUPIED
13      if match then match.status ← CHECKED_IN
14      create session(ACTIVE, plate, bay, match, evidence)
15      write detection_event(plate, confidence, candidates, decision)
16      broadcast slot.allocated
17      return {status: allocated, slot: bay, open_barrier: true}
```

Three properties of this algorithm are worth stating explicitly. First, the booked bay wins over the lowest-numbered bay: step 10 reads the booking's slot identifier before falling back, which is the fix for a real defect in which the driver was sent to a different bay from the one reserved (D-01 in Table 16). Second, duplicate frames are suppressed twice, once by the active-session check and once by a ten-second per-plate cooldown, because the camera posts every five seconds and a stationary car at the gate would otherwise generate two allocations. Third, the walk-in path is a policy switch, not a separate endpoint: an unreserved vehicle is admitted only when `ALLOW_WALKIN_ENTRY` is active, and it draws only from bays that are actually free, so it can never take a bay that is held by a booking.

7.4 Exit release and billing algorithm

```
Algorithm 3  ReleaseExit(plate, confidence, candidates)
 1  if confidence < 0.75 then return ignored
 2  matches ← ACTIVE sessions scoring ≥ 0.72 against the candidates
 3  if matches is empty then return no_active_session
 4  if top two scores are within 0.10 then return ambiguous
```

```
  5  session ← the unique winner
  6  session.status ← COMPLETED; session.exit_time ← now (server UTC)
  7  bay.status ← AVAILABLE; booking.status ← COMPLETED
  8  d ← max(1, floor((exit_time − entry_time) / 60))          # whole minutes
  9  amount ← 30 if d ≤ 60 else 30 + ((d − 1) div 60) × 20
 10  write detection_event(plate, confidence, candidates, decision)
 11  broadcast slot.released, vehicle.exited, barrier.exit
 12  return {status: released, slot, duration_minutes: d, billed_amount: amount,
             open_barrier: true}
```

The billing formula is the only place in the system where money is computed, and it is computed on the server so that the receipt cannot be edited by the client. The tariff is ₹30 for the first hour and ₹20 for every additional started hour; a stay of sixty minutes bills ₹30, sixty-one minutes bills ₹50, and one hundred and twenty-one minutes bills ₹70. Duration is floored to whole minutes and clamped to a minimum of one, so a vehicle that enters and leaves within the same minute is still billed for one minute rather than zero.

7.5 Decision flow chart

Figure 3 shows the decision half of the pipeline as a flow chart. The three outcomes on the right — allow, deny and hold — correspond exactly to the three physical behaviours of the barrier: open, remain closed, and remain closed with an operator alert. The chart is drawn with a single decision point for each gate so that the order of the gates is visible: quality, detection, reading, scoring, voting, matching, and finally the choice between allow, deny and hold.

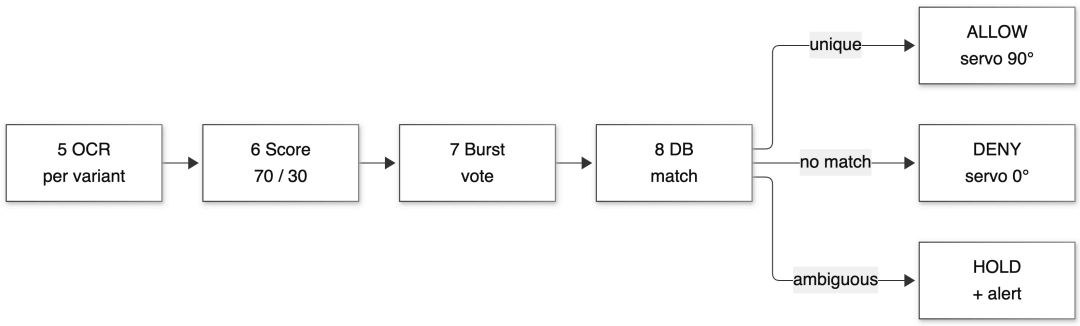


Fig. 3 : Decision path, stages 5–8, with allow / deny / hold outcomes

7.6 Data-flow view

At level 0 the system is a single process that receives frames from two cameras and commands from two clients, reads and writes one database, and returns barrier commands and state to the same parties. At level 1 the process divides into four stores and five transformations: the frame store (transient), the session store, the booking store and the detection-event store; and the transformations *assess*, *read*, *match*, *allocate* and *release*. The context diagram of Fig. 1 is the level-0 view; the component view of Fig. 7 in Chapter 9 is the level-1 view. The important data-flow property is that no transformation writes to the session store without also writing a detection event, so the audit trail cannot diverge from the state.

7.7 State machines

Three state machines define the legal transitions in the system. They are small enough to be checked by inspection, and together they encode the invariants of Section 5.8.

Parking bay. A bay is created AVAILABLE. A pre-booking moves it to ALLOCATED and holds it. A matched entry moves it to OCCUPIED; an exit match, an operator release or a facility reset returns it to AVAILABLE. A bay can be moved to BLOCKED for maintenance and back. No transition into OCCUPIED is possible without a session, and no transition out of OCCUPIED is possible while a session is active.

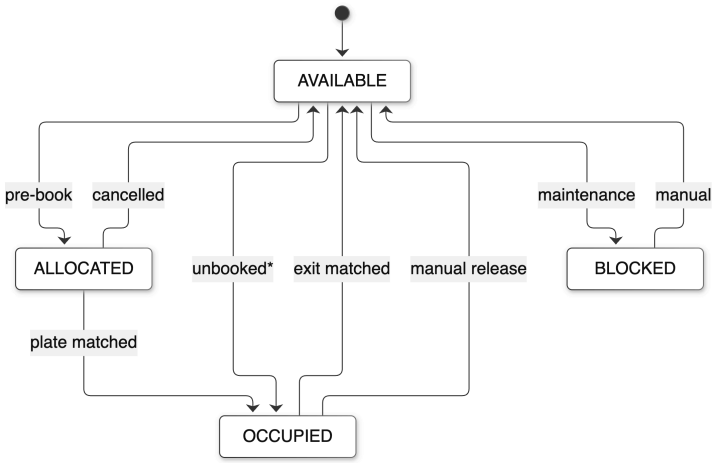


Fig. 4 : Parking bay state machine

Booking. A booking is created PENDING_ARRIVAL with a bay and an expiry time. It becomes CHECKED_IN when the vehicle arrives, CANCELLED when the driver cancels, EXPIRED when the expiry passes without arrival, and COMPLETED when the corresponding session closes. Every terminal state releases the bay, which is what prevents the phantom occupancy defect recorded as D-02.

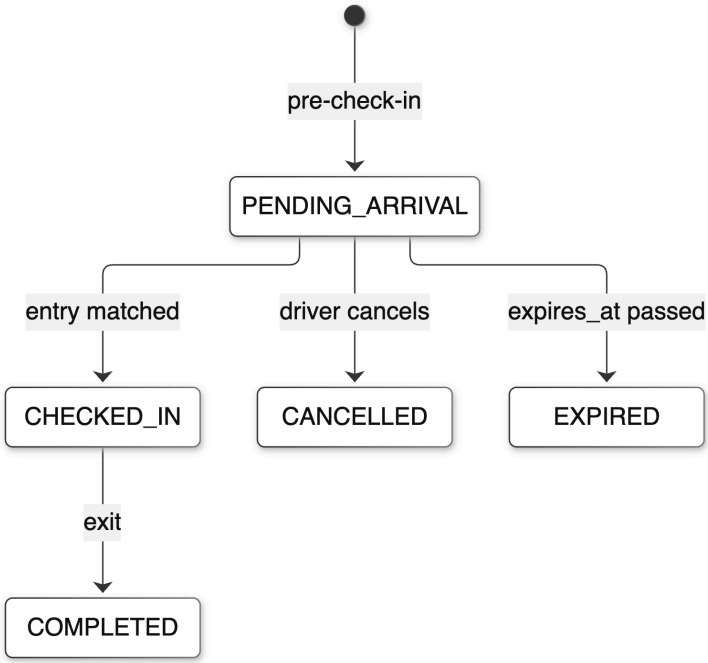


Fig. 5 : Booking state machine

Session. A session is created ACTIVE at entry and becomes COMPLETED at exit or on an operator reset. There is no suspended state and no re-open path, which keeps the occupancy query trivial: a bay

is occupied if and only if there exists a row with status ACTIVE that references it.

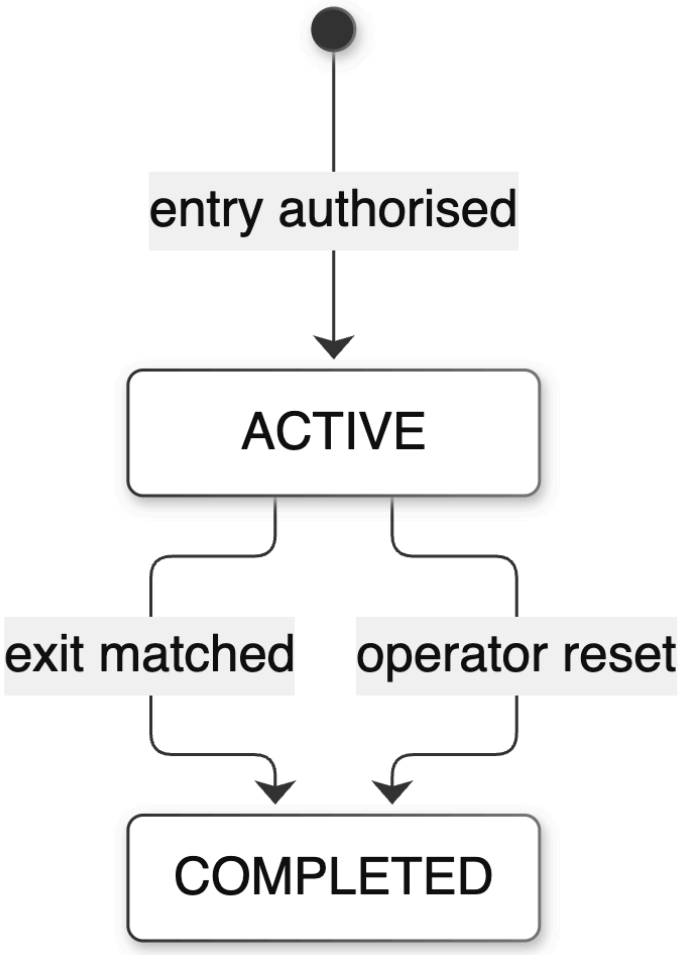


Fig. 6 : Parking session state machine

7.8 Worked examples

Canonical match. A booking exists for KA05KK0403. The camera reads KA05KK04O3, with the letter O in place of the final zero. Both strings canonicalise to KA05KK0403 through the confusion map, so the similarity is 0.92. The OCR confidence for the candidate is 0.91, so the adjusted score is 0.92 × (0.80 + 0.20 × 0.91) = 0.92 × 0.982 = 0.90, which is above the 0.72 threshold. The vehicle is admitted and the attendance is recorded against the booking.

Ambiguity refusal. Two bookings exist, for KA05KK0403 and KA05KK0404. The camera reads KA05KK0405, which is one edit from each. Both score 0.85, the gap is 0.00, and the ambiguity rule fires: the barrier stays closed and the operator sees both candidate plates. The system has not guessed, and the driver is asked to present the plate again rather than being sent to a bay that may belong to someone else.

Billing boundaries. A stay of 60 minutes bills ₹30; 61 minutes bills ₹50; 120 minutes bills ₹50; 121 minutes bills ₹70. The boundary behaviour is deliberate: the first hour is a block, and each additional started hour is a block, so the charge never decreases as the stay grows and never charges a partial hour as a full one.

CHAPTER 8

Working Principle of the Project

This chapter explains, in one continuous narrative, how the system works: what happens between the moment a vehicle approaches a gate and the moment the barrier moves, and why each step is necessary. The algorithms of Chapter 7 give the precise order of operations; this chapter gives the mechanism and the reasoning behind it.

8.1 The operating principle in one sentence

A parking bay is free if and only if a vehicle has entered the facility and has not yet left it. Everything else follows from that sentence. The entry gate is the only place where a vehicle can begin to occupy a bay, the exit gate is the only place where it can stop occupying one, and the database — not a sensor inside the lot — is the record of both events. The two cameras are therefore not measuring parking spaces; they are observing the boundary of the facility, and the occupancy of every space is derived from those boundary observations.

8.2 From photons to a decision

The entry node wakes on a five-second cycle, captures a 640×480 JPEG and posts it as a multipart request to `/api/detection/entry`. The request carries no decision and no state; the node does not know whether the vehicle is expected. On the server, the frame is first graded. The variance of the Laplacian measures sharpness, the mean pixel intensity measures exposure, and the standard deviation measures contrast; a frame that is too blurred, too dark or too flat is labelled and the pipeline stops there, because running a detector on a frame that cannot possibly yield a readable plate wastes two seconds of OCR and produces a misleading confidence. The grade is recorded with the frame, so a refusal caused by the camera can be distinguished from a refusal caused by an unknown plate.

If the frame passes the quality gate, the plate detector returns a bounding box and a confidence. The crop inside the box is then normalised in four different ways — adaptive threshold, inverted adaptive threshold, Otsu threshold and CLAHE — because no single binarisation works across the lighting conditions a gate camera sees in a day. Each variant is passed to the OCR engine with an alphabet restricted to capital letters and digits, which removes punctuation noise, and the results are sorted top-to-bottom and then left-to-right so that a two-line plate concatenates in the right order. The candidates are scored for confidence and for agreement between variants, and the burst accumulator merges them with candidates from the previous frames in the same five-second window.

8.3 The entry decision

The highest-scoring candidate is then compared with the bookings that are waiting for arrival. The comparison is not a string equality test: both strings are canonicalised through a map of visually confusable characters, the residual distance is measured, and the resulting similarity is discounted by the OCR confidence. A booking matches if the adjusted score is at least 0.72. If exactly one booking matches, the vehicle is admitted to the bay that the booking holds. If no booking matches and walk-in admission is enabled, the vehicle is admitted to the lowest-numbered free bay, and the bay is highlighted on the dashboard so the driver can see where to go. If two bookings match within 0.10 of

each other, the system refuses and raises an alert. If no bay is free, the status is `parking_full` and the display says the lot is full rather than that the vehicle is unknown, because those are different problems for the driver.

Admission is a single database transaction: the bay becomes OCCUPIED, the booking becomes CHECKED_IN, a session becomes ACTIVE, and a detection event is written with the plate, the confidence, the candidate list and the decision. Only after the transaction commits does the server return `open_barrier: true`. The firmware reads that boolean and, if it is true, rotates the servo to 90°, waits five seconds and returns it to 0°. If the response is delayed, malformed, or reports any other status, the arm does not move. This ordering matters: the barrier is a consequence of a committed decision, never a cause of one.

8.4 The exit decision

The exit path is the mirror image, with one difference in consequence. The frame is read in the same way, but the candidates are compared against ACTIVE sessions rather than pending bookings. A unique match closes the session, records the exit time, frees the bay, completes the booking, computes the duration in whole minutes and applies the tariff. The response carries the released bay, the duration and the amount, and the client renders an itemised receipt. If the plate matches no active session, the barrier stays closed and the operator is alerted. If it matches two, the system holds rather than guessing, because releasing the wrong bay would free a space that is still occupied.

The asymmetry is deliberate. At entry, a false admit puts a vehicle into a bay that may belong to someone else; at exit, a false release frees a bay that is still occupied and produces a bill for the wrong vehicle. Both corrupt the same derived state, and both are prevented by the same rule: the system acts only when the evidence resolves to exactly one interpretation.

8.5 Why the principle works

The principle works because the two events that change occupancy are exactly the two events that a gate can observe reliably. A vehicle cannot enter a lot without passing the entry gate, and it cannot leave without passing the exit gate; every other movement inside the lot is irrelevant to the count. This is the same reasoning that makes a ticket-based car park work, with the ticket replaced by a plate read. It is also why the architecture can be constant in cost: the number of observations needed is two per vehicle movement, regardless of how many bays the movement affects.

The principle also has a known cost, and the system is designed around it rather than pretending it does not exist. If a vehicle leaves and its plate is never read — because the camera was blinded by headlights, or the plate was dirty, or the vehicle left during a network outage — the bay remains occupied until a person intervenes. The manual release path exists for exactly this case, it is available in the operator console, and it writes an audit entry. A per-bay sensor would not have this failure mode. The project's claim is not that the derived state is as robust as a measurement; it is that the derived state is accurate enough for a small lot, that its failure mode is visible and recoverable, and that it removes a cost that grows with the size of the lot.

8.6 Behaviour under failure

Because the barrier is fail-closed, the failure of any component between the camera and the database results in a closed gate rather than an open one. A camera that loses Wi-Fi keeps retrying every five

seconds and the barrier stays down. A backend that is restarted loses no lot state, because the state is in SQLite, and the cameras resume posting as soon as the service answers. A browser that disconnects re-reads a snapshot on reconnect and then resumes the stream, so a laptop that sleeps does not leave a stale occupancy map on screen. The operator console reports a camera as offline after thirty seconds without a frame, which is long enough to survive a dropped packet and short enough to be noticed during a demonstration.

CHAPTER 9

Proposed Block Diagram and Circuit Diagram

This chapter presents the system as a set of blocks and then at the level of a single gate node. The block diagram shows what talks to what and, more importantly, where the state lives; the circuit diagram shows how one ESP32-CAM is wired to its servo, its buzzer and its supply, because the same wiring is repeated at both gates.

9.1 Block diagram

Figure 7 divides the system into an edge block, an application block, a data block and a presentation block. The arrows carry either a frame (camera to server), a command (server to actuator), a query (client to server) or an event (server to client). No arrow carries state from a camera to another camera, and no arrow carries occupancy from the server to the database: the database is written by the server and read by the server, and the clients only ever read a projection of it.

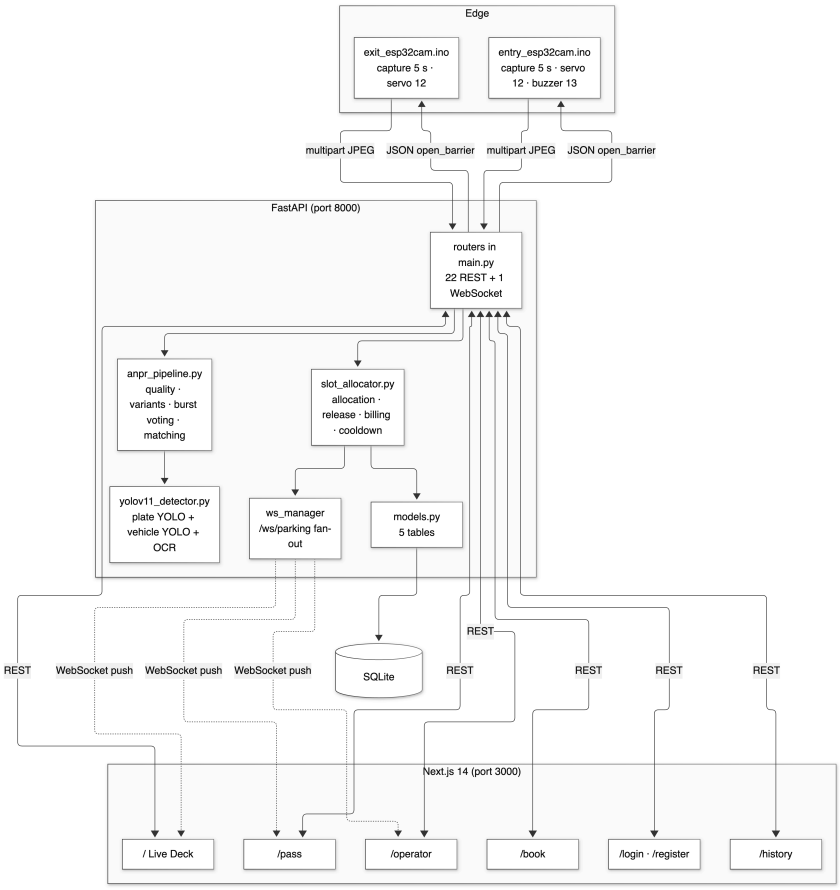


Fig. 7 : Block diagram of the proposed system

1. **Edge block, entry node.** ESP32-CAM with an OV2640 sensor. Captures a VGA JPEG every five seconds, posts it to the entry detection route, parses the JSON response and drives the entry servo. Holds no lot state.
2. **Edge block, exit node.** Identical hardware and firmware without the buzzer. Posts to the exit detection route and drives the exit servo.

3. **Routing block.** The FastAPI application in `main.py`: twenty-two REST routes and one WebSocket route, request validation, dependency injection of the database session and the current user, and the broadcast calls that publish every state change.
4. **Recognition block.** The ANPR pipeline and the YOLO/OCR detector wrapper: quality gate, four preprocessing variants, candidate scoring, burst voting and the two matching functions for entry and exit.
5. **Policy block.** The slot allocator: allocation, release, billing, cooldown, manual release and reset. This is the only block that changes the status of a bay.
6. **Data block.** SQLite through SQLAlchemy, five tables: users, parking slots, pre-registrations, sessions and detection events.
7. **Fan-out block.** The WebSocket manager, which keeps the set of live sockets and broadcasts typed events without blocking the request path.
8. **Presentation block.** The Next.js client: the Live Deck, the pre-booking page, the active pass, the receipts page and the operator console. It renders state and never computes occupancy.

9.2 Circuit diagram of a gate node

Figure 8 shows the electrical connections for one gate node. The ESP32-CAM is powered from the 5 V rail; the servo is powered from the same rail but through its own pair of leads, with the signal wire on GPIO 12 and the grounds tied together. The optional buzzer sits on GPIO 13 through a current-limiting resistor. The capacitor across the 5 V rail is not decorative: it supplies the surge current when the servo starts moving and prevents the brown-out reset that otherwise looks like a firmware crash.

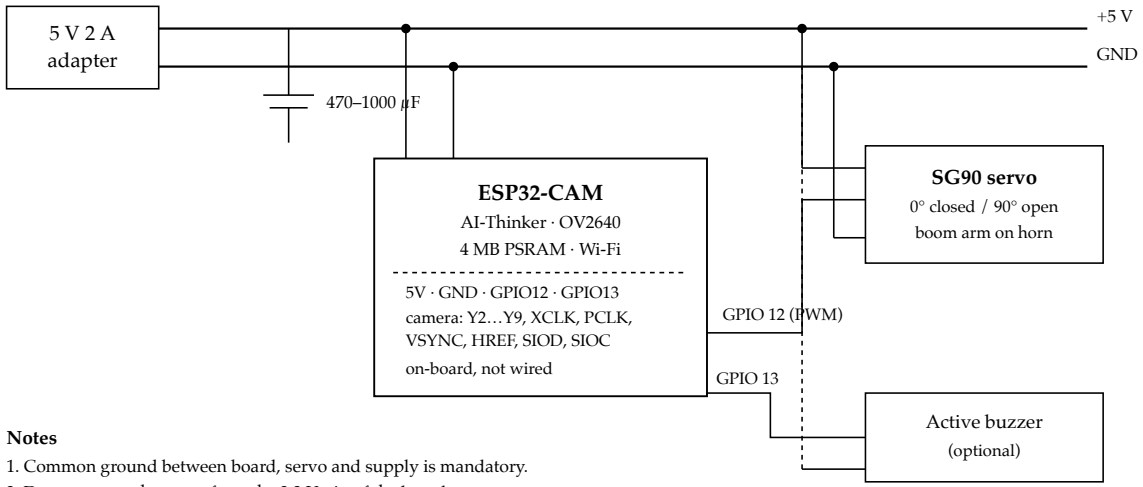


Fig. 8 : Circuit diagram of one gate node

9.3 Pin map

Table 7 gives the pin assignment for both nodes. GPIO 12 and GPIO 13 are used because they are free on the AI-Thinker camera pinout: most other pins are consumed by the camera bus or by the flash

interface, and choosing a pin that looks free in a generic ESP32 diagram is the most common wiring mistake on this board.

Signal	ESP32-CAM pin	Notes
Servo signal (orange / yellow)	GPIO 12	PWM control; 0° closed, 90° open
Servo VCC (red)	5 V rail	Not 3.3 V — brown-outs and jitter otherwise
Servo GND (brown / black)	GND	Must share ground with the ESP32-CAM
Buzzer + (optional)	GPIO 13	Active buzzer; disabled by setting the pin to −1
Buzzer −	GND	—
Camera	Internal AI-Thinker header	Y2…Y9, XCLK, PCLK, VSYNC, HREF, SIOD, SIOC, PWDN, RESET

Table 7 : Pin map of a gate node

9.4 Wiring tables

Tables 8 and 9 give the point-to-point wiring for the two nodes. The only difference between them is the buzzer, which is present at the entry gate so that a driver receives an audible confirmation, and omitted at the exit gate to keep the departure quiet.

From	To	Purpose
ESP32-CAM 5V	5 V rail	Board power
ESP32-CAM GND	Common GND	Reference for the PWM signal
GPIO 12	Servo signal	Barrier PWM
5 V rail	Servo VCC	Torque supply
Common GND	Servo GND	Return
GPIO 13	Buzzer +	Confirmation chirp (optional)
GND	Buzzer −	Return

Table 8 : Wiring table — entry node

From	To	Purpose
ESP32-CAM 5V	5 V rail	Board power
ESP32-CAM GND	Common GND	Reference for the PWM signal
GPIO 12	Servo signal	Barrier PWM
5 V rail	Servo VCC	Torque supply
Common GND	Servo GND	Return

Table 9 : Wiring table — exit node

9.5 Power topology

The supply is a single 5 V 2 A adapter per node, distributed to the board and to the servo. The board's regulator supplies the camera; the servo is fed directly from the rail. The grounds are common. A 470–1000 μF electrolytic capacitor sits across the rail close to the servo to absorb the inrush when the arm starts moving. The measured consequence of omitting the capacitor is visible on the bench: the board resets at the moment the servo starts, the serial log restarts, and the barrier occasionally twitches

instead of sweeping. It is a power problem that presents as a software problem, and it is listed first in the troubleshooting guide in the appendix.

9.6 Barrier control contract

The barrier is controlled by one boolean in the response, not by an action verb. The complete contract for an admitted entry is shown below; the exit response has the same shape with the released bay, the duration and the amount.

```
{
  "status": "allocated",
  "slot": "A2",
  "open_barrier": true,
  "barrier_action": "OPEN",
  "trigger_buzzer": true
}
```

The firmware decision is equally small. Anything other than an explicit true leaves the arm where it is:

```
bool open_barrier = doc["open_barrier"] | false;
if (open_barrier || trigger_buzzer) {
    openEntryBoomBarrier();      // 90 degrees -> 5000 ms -> 0 degrees
} else {
    // gate stays at 0 degrees
}
```

Table 10 states the outcomes for every status the backend can return. The table is the specification against which the firmware is reviewed, and it is the reason the system can be described as fail-closed: there is no row in which an error, a missing field or an unknown status results in an open barrier.

Backend outcome	open_barrier	Entry barrier	Exit barrier	Operator sees
allocated (valid booking)	true	OPEN 90° / 5 s	—	Bay number, plate, confidence
released (valid session)	true	—	OPEN 90° / 5 s	Duration and amount
unauthorized	false	closed 0°	—	Plate plus alert
no_plate	false	closed 0°	—	Failure stage and quality grade
ambiguous	false	closed 0°	closed 0°	Both candidate plates
no_active_session	false	—	closed 0°	Plate plus alert
parking_full	false	closed 0°	—	Full-lot alert
already_allocated	false	closed 0°	—	Existing bay for that plate
timeout / malformed / unknown	absent	closed 0°	closed 0°	Camera marked offline after 30 s

Table 10 : Barrier control truth table

CHAPTER 10

Proposed Methodology

This chapter describes how the work was carried out: the phases, the development method, the data design, the way the system was verified, and the protocol that must be followed to turn the recognition results from observations into measurements. The method was iterative rather than waterfall, but the phases are described in the order in which they were completed so that the dependencies between them are visible.

10.1 Development method

The project used a short-cycle iterative method. Each cycle addressed one vertical slice of the system — recognition, then allocation, then billing, then the operator view — and ended with the slice running end to end against the real database, not against a mock. This was chosen because the integration points were the risky part: a detector that works on a still image and an allocator that works on a test fixture can still fail when a real frame arrives while a booking is expiring. Building vertically meant that every week produced something demonstrable, and that the defect log grew from real integration rather than from code review.

Two practices were adopted from the start and are visible in the repository. The first is a single source of truth for state: no component was allowed to cache occupancy, and every screen reads the server. The second is that every change to the allocation or billing logic had to be accompanied by a check in the automated suite, which is why the suite is written around the eight scenarios of Table 14 rather than around units of code.

10.2 Phases and deliverables

Table 11 lists the phases, the main activities in each, and the artefact that was produced. The phase boundaries are not arbitrary: each one ends with something that can be inspected or executed by someone other than its author.

Phase	Focus	Main activities	Deliverable
P1	Problem definition	Field observation, literature survey, scope negotiation with the guide	Problem statement, objectives, scope boundary
P2	Architecture	Layer selection, state ownership, API surface, schema design	Context and component diagrams, database schema
P3	Recognition	Dataset collection, YOLO fine-tuning, variant design, OCR scoring, burst voting	Working pipeline with per-stage diagnostics
P4	Authorisation and allocation	Booking model, matching rules, allocator, cooldown, walk-in policy	Allocator with eight decision statuses
P5	Billing and audit	Tariff implementation, session closure, detection-event log	Receipt path and evidence table
P6	Client	Live deck, booking, pass, receipts, operator console, design tokens	Eight-page Next.js application
P7	Hardware	Node assembly, servo calibration, power topology, firmware	Two gate nodes and the barrier contract
P8	Verification	Automated suite, interface checks, defect fixing, regression checklist	15/15 automated checks, defect log

Phase	Focus	Main activities	Deliverable
P9	Evaluation	Diagnostic capture, metric definitions, measurement protocol	Results chapter and pending-measurement list
P10	Documentation	Report, diagrams, appendix, demonstration script	This report

Table 11 : Methodology phases and deliverables

10.3 Data design

The data model was designed before the recognition pipeline, because the decision the pipeline has to support is a database question: does this plate correspond to exactly one pending booking or active session? Five tables are enough. Figure 9 shows the entity–relationship diagram. The session table is the occupancy authority; the detection-event table is the evidence authority; the booking table is the reservation authority; and the slot table is the inventory. The user table exists only to scope bookings and receipts to an account.

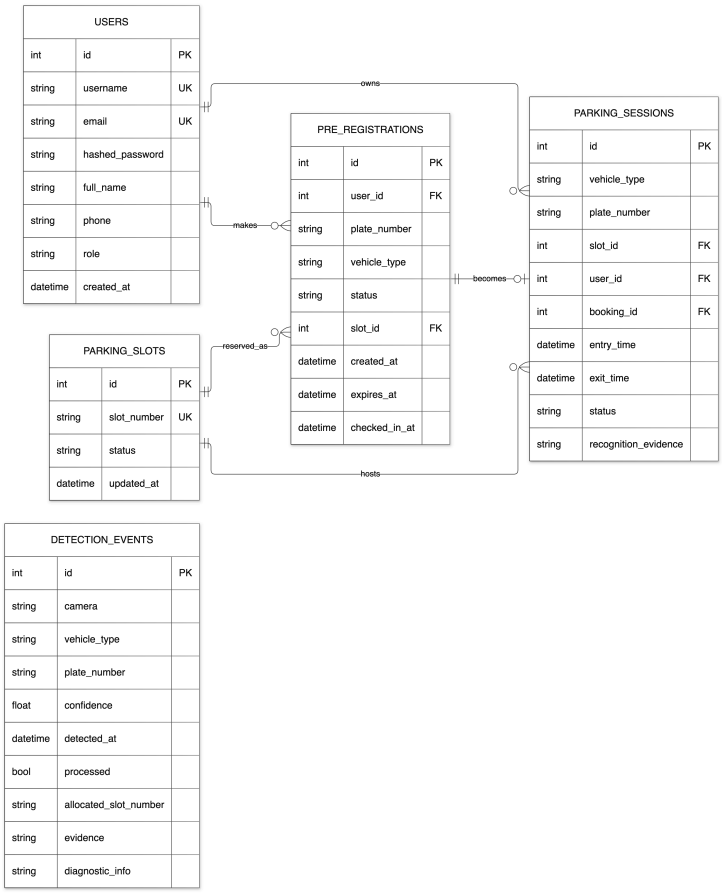


Fig. 9 : Entity–relationship diagram of the five-table schema

Three schema decisions are worth recording. The evidence column on a session stores the recognition evidence as a JSON string rather than a normalised structure, because it is written once and read for audit rather than queried by field. The booking carries a concrete slot identifier rather than a zone preference, because the driver is promised a specific bay and a promise that names a zone is not a promise. And the detection-event table stores the failure stage even when no plate was found, which

is what makes it possible to say afterwards whether a refusal was caused by the camera, the detector or the matcher.

10.4 Verification strategy

Verification was organised in six levels, from a unit check of the billing formula to a dry run of the demonstration script. The automated suite starts its own backend on an isolated port with a temporary SQLite file, so it can be run during development without disturbing the demonstration data. The suite is deliberately written against the HTTP surface rather than against internal functions, because the contract between the firmware and the server is the part that has to be right at the gate. The levels and the results are reported in Chapter 12.

10.5 Measurement protocol

Recognition accuracy and latency are properties that must be measured, not asserted, and they have not yet been measured on this build. The protocol is fixed here so that the numbers, when they are produced, are comparable and honest. It follows the usual practice in the ANPR literature of reporting per condition rather than as a single average [3].

Sampling. Use at least five distinct plates, including two that differ by one character, so that the ambiguity rule is exercised. For each plate, capture frames under four conditions: daylight frontal, low light, angled at about 30°, and motion blur.

Volume. At least ten frames per plate–condition pair, giving at least two hundred frames in total, with a ground-truth CSV recording the frame, the true plate and the condition.

Execution. Post each frame to the detection endpoint with a known booking state, or run the pipeline offline, and log the status, the selected plate, the confidence and the failure stage for every frame.

Metrics. Plate detection rate (frames with a plate box divided by frames containing a plate); exact recognition rate (readings exactly equal to ground truth divided by frames with a detected plate); matcher-accepted rate (readings accepted as the correct vehicle divided by frames with a detected plate); false admit and false reject rates over the decision outcomes; and end-to-end latency from the start of the POST to the JSON response, reported as p50 and p95 over at least thirty runs with the hardware stated.

Honesty rule. A metric that has not been produced by this protocol is reported as not measured, not as an estimate. The two diagnostic records in Chapter 12 are single observations reproduced from the bench; they are labelled as such and are not presented as a benchmark.

10.6 Risk management

The principal risks were recognition quality on a low-cost camera, power instability at the servo, and dependency drift in the Python environment. The first was mitigated by designing the refusal path before tuning the recognition pipeline, so that a poor read produces a defined refusal rather than an undefined failure. The second was mitigated by separating the servo supply and adding the capacitor, and by writing the bench checklist in the appendix. The third was mitigated by pinning the backend interpreter in the start script; the remaining packaging gap (easyocr and ultralytics absent from `requirements.txt`) is recorded as an open defect rather than dismissed.

CHAPTER 11

Overall Working of the Project

This chapter follows the system through the five flows that constitute its working life — a pre-booking, an entry by a booked vehicle, an entry by an unbooked walk-in vehicle, an exit, and an operator intervention — and then places those flows on the network and inside the gate firmware. The sequence diagrams are not schematic: each message corresponds to a route or a database write that exists in the repository, and the alternative blocks correspond to the refusal branches that Chapter 8 described.

11.1 The five flows at a glance

Every operation in AutoPark is a variation of one pattern: a client or a camera sends evidence to the server, the server reads the current state of the lot, applies one policy, writes one transaction, publishes one event and replies with an explicit outcome. There are five such flows. The *booking flow* binds a bay to a plate before the vehicle arrives. The *booked-entry flow* consumes that booking and turns it into a session. The *walk-in entry flow* creates a session without a booking, taking the lowest-numbered free bay. The *exit flow* closes a session, frees the bay and produces the bill. The *operator flow* is the exception path: it releases a bay, resets the demonstration, or inspects why a decision was taken. No other flow can change the occupancy of a bay, which is what makes the occupancy invariant checkable by inspection.

11.2 Booking flow

A driver selects a bay, a plate, a vehicle category and an expected duration on the pre-booking page. The client posts the request with the driver's bearer token to `/api/bookings/pre-checkin`. The server normalises the plate, rejects it if it has fewer than four characters, rejects the request if the driver already has an active session, rejects it if the same plate is already inside the lot, and resolves the bay: the requested bay if it is still free, otherwise the lowest-numbered free bay. The bay is marked ALLOCATED and the booking is written with status PENDING_ARRIVAL and an expiry of the requested duration. Two events are published — `booking.created` and, if the bay changed, `slot.released` for the bay that was given up — and the response returns the bay number that the driver is promised.

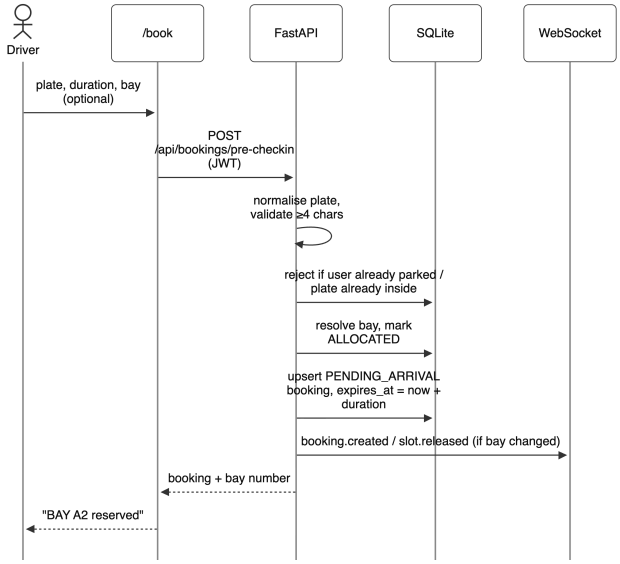


Fig. 10 : Booking sequence — reservation before arrival

The visible result is the pass screen, which names the bay, its zone and its location within the lot. From that moment the bay is not offered to any other driver, and the entry allocator will bind that specific bay rather than the next free one. This is the flow that makes Objective 3 testable: the promise is a bay identifier, not a zone.

11.3 Booked entry flow

The entry node captures a frame, posts it to `/api/detection/entry` and waits. On the server the frame passes the quality gate, the detector locates the plate, the four preprocessing variants are read and the burst vote produces a ranked candidate list. The candidates are compared with every booking that is still PENDING_ARRIVAL. If exactly one booking scores at least 0.72 and is separated from the runner-up by 0.10, the allocator binds the booking's bay, marks the bay OCCUPIED, marks the booking CHECKED_IN, creates an ACTIVE session and writes a detection event. The response carries `status: allocated`, the bay, and `open_barrier: true`. The firmware reads that boolean, sweeps the servo to 90°, holds for five seconds and returns to 0°.

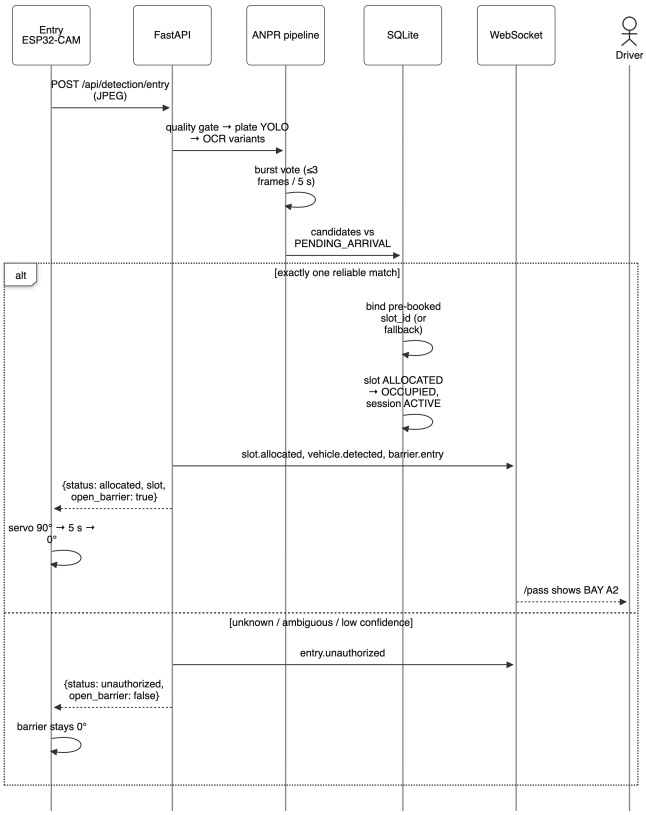


Fig. 11 : Entry sequence — authorised entry and refusal paths

The lower half of Fig. 11 is the branch that matters most. If the plate is unreadable, unknown, or matches two bookings, the status is `no_plate`, `unauthorized` or `ambiguous` respectively, `open_barrier` is false, and the firmware does not move. The event published on the WebSocket channel carries the plate, the confidence and the failure stage, so the operator console shows the reason rather than a generic denial. The whole branch is one code path with three exit labels, not three separate implementations, which is why a defect in the refusal logic cannot affect only one status.

11.4 Walk-in entry flow

A walk-in vehicle is mechanically identical to a booked one and differs in a single decision: when no booking matches, the allocator consults the walk-in policy. If admission is enabled and a bay is free, the lowest-numbered free bay is allocated, the bay is marked OCCUPIED, a session is created with no booking reference, and the response names the bay so that the driver can be told where to go. The live dashboard highlights the newly allocated bay and the arrival callout names the plate and the bay, which is what allows an unattended lot to route a spontaneous arrival without a printed ticket. If the walk-in policy is disabled the same request returns `unauthorized`, and if no bay is free it returns `parking_full` — a different message, because the driver's problem is different.

The policy switch is a deliberate operational choice rather than a hidden default. A facility that promises every bay to advance bookings sets it off; a department lot that must not turn away staff sets it on. The setting is read on every request, so it can be changed without a restart, and the audit row records which path was taken.

11.5 Exit flow

The exit node posts to `/api/detection/exit` in exactly the same way. The candidates are now compared with ACTIVE sessions instead of pending bookings. A unique match closes the session, stamps the exit time, returns the bay to AVAILABLE, completes the booking if there was one, computes the duration in whole minutes and applies the tariff. The response carries the released bay, the duration, the amount and `open_barrier: true`; the client renders the receipt, and the WebSocket publishes `slot.released`, `vehicle.exited` and `barrier.exit` so that the dashboard and the operator console update together.

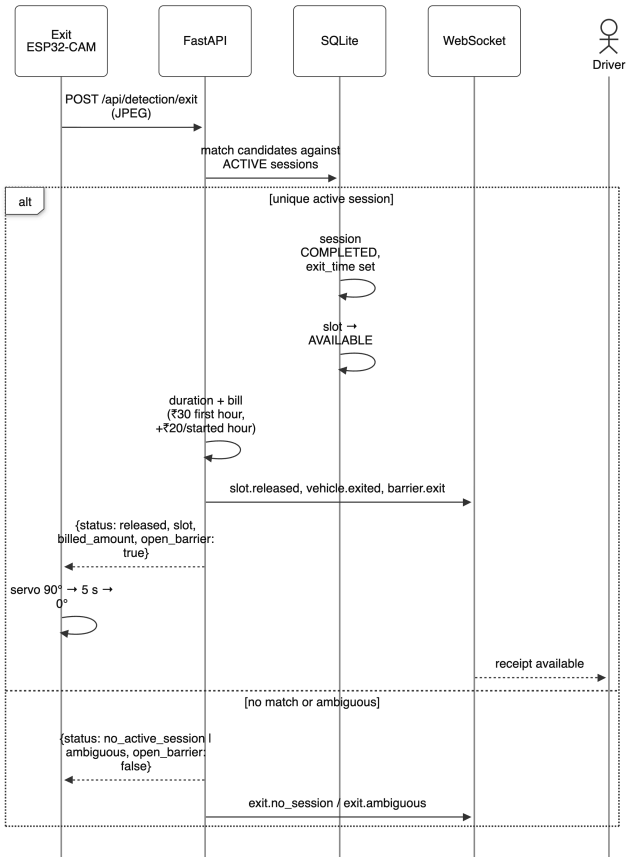


Fig. 12 : Exit sequence — release and billing

If the plate matches no active session the status is `no_active_session` and the barrier stays down; if two sessions are equally close the status is `ambiguous` and the operator is alerted. Neither case produces a bill, because a bill is only ever produced by a closed session. The one recovery path that the design requires is the operator release described next.

11.6 Operator exception handling

Three situations require a human. First, a vehicle whose exit plate cannot be read leaves a session open; the operator sees the bay as occupied, verifies the vehicle physically, and releases the bay from the console, which closes the session and writes an audit entry naming the operator action. Second, a driver whose entry was refused needs to know why; the console shows the failure stage, the quality grade and the candidate list from the diagnostic record, so the operator can distinguish a dirty plate from an unpaid booking. Third, a demonstration must start from a known state; the reset route closes

every session, returns every bay to AVAILABLE, clears cooldowns and publishes `demo.reset`. All three actions are available in the console, all three are idempotent, and all three publish events, so no screen is left showing a state that no longer exists.

11.7 Deployment topology

Figure 13 shows the deployment. Everything runs on one facility LAN with no cloud dependency. The Next.js client listens on port 3000 and talks to the API on port 8000 over REST for queries and over a WebSocket for events. The API owns the SQLite file and is the only process that writes to it. The two ESP32-CAM nodes are ordinary LAN clients: they post JPEGs to the detection routes and receive JSON. The host laptop is therefore the whole server tier, which is adequate for a six-bay pilot and is also the largest single point of failure in the system; the report treats that as a deployment limitation rather than pretending it is a fault-tolerant design.

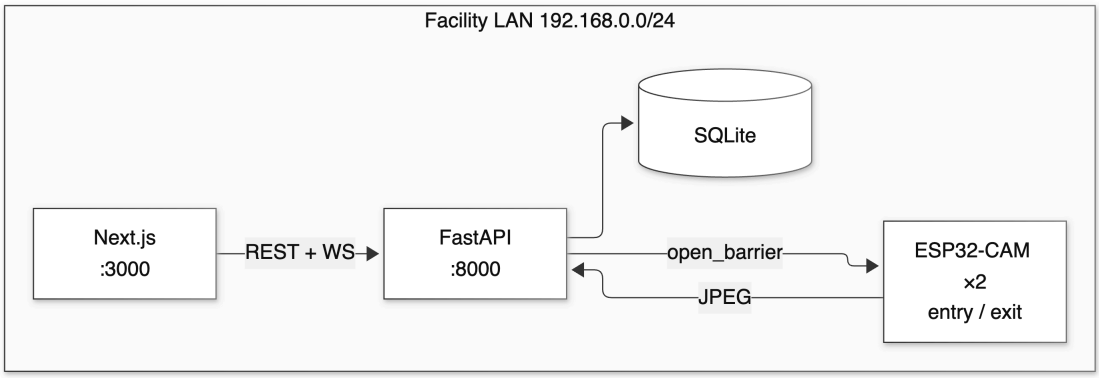


Fig. 13 : Deployment topology on the facility LAN

11.8 Firmware lifecycle

The firmware is a loop with one decision in it. On power-up the node sets the servo to 0° so that the gate is closed before anything else happens, connects to Wi-Fi with a bounded retry count, initialises the camera and enters the capture cycle. Each cycle captures a frame, posts it with a fifteen-second timeout, parses the JSON and asks one question: is `open_barrier` true? If it is, the arm sweeps to 90°, waits five seconds and returns. If it is not — because the field is false, because the response was malformed, because the request timed out, or because the server could not be reached at all — the arm stays at 0° and the node logs the outcome and waits for the next cycle. Camera initialisation failure is handled explicitly: the node reports the fault on the serial line and halts capture rather than posting blank frames, so a failed camera cannot be mistaken for an empty lane.

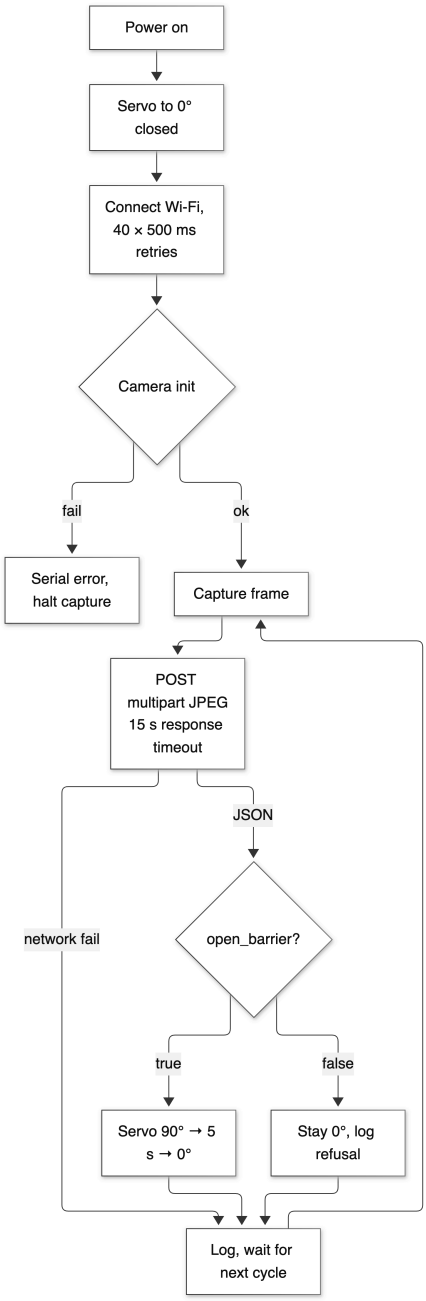


Fig. 14 : Firmware lifecycle of a gate node

The consequence is a device that cannot open a gate by itself. It has no notion of a booking, a bay, a tariff or a session, and it holds no state across cycles. If the server is upgraded, restarted or disconnected, the nodes keep capturing and posting, and every gate remains closed until the server answers with an explicit true. That is the intended direction of failure for an access-control device, and it is the property that Objective 5 is written against.

11.9 Real-time event flow

Every transition is published on the single channel `/ws/parking`. Clients open the socket on mount, patch their local state from each event, and re-read a snapshot after any reconnect. Table 12 lists the event types that the server publishes and the screen that consumes each one. The list is short on

purpose: an event vocabulary that grows without limit is a symptom of state being invented in the client.

Event type	Published when	Payload essentials	Consumed by
`slot.allocated`	A bay is bound to a session at entry	Slot, plate, session id	Live Deck, pass, operator plan
`slot.released`	A session closes or a booking gives up a bay	Slot, plate	Live Deck, operator plan
`vehicle.detected`	A frame yields a plate candidate above threshold	Camera, plate, confidence	Operator camera panels
`vehicle.exited`	An exit match closes a session	Plate, duration, amount	Receipts, operator
`entry.unauthorized`	An entry is refused for any policy reason	Plate, status, failure stage	Operator alerts
`exit.no_session`	An exit plate matches no active session	Plate, candidates	Operator alerts
`exit.ambiguous`	An exit plate matches two sessions	Both candidate plates	Operator alerts
`parking.full`	An entry is refused because no bay is free	Lot counters	Live Deck banner
`barrier.entry`	The entry barrier is commanded open	Slot, plate	Operator gate panel
`barrier.exit`	The exit barrier is commanded open	Slot, plate	Operator gate panel
`buzzer.triggered`	An entry is admitted with audible confirmation	Camera	Operator gate panel
`booking.created`	A pre-booking is accepted	Booking id, slot, plate	Pass screen, operator
`booking.cancelled`	A booking is cancelled by the driver	Booking id, slot	Pass screen, operator
`demo.reset`	The facility is reset between demonstrations	Counters	All screens

Table 12 : WebSocket event catalogue

Two details make the channel safe to rely on. The server broadcasts after the transaction commits, never before, so a client that receives an event can be certain that a subsequent REST read will agree with it. And a client that reconnects re-reads the layout and summary before applying further events, so a missed message cannot leave a stale bay on screen. The second rule is the one that saves the demonstration when a laptop sleeps and wakes.

CHAPTER 12

Results (Simulation / Experimental) and Justifications

This chapter reports what was observed, what was measured and what was not. The distinction is kept explicit throughout, because the most common weakness of a student report in this area is a single successful demonstration presented as a benchmark. Section 12.1 defines the six levels at which the system was checked; Sections 12.2–12.7 report the results at each applicable level; Section 12.8 gives the engineering metrics that can be counted exactly; Section 12.9 gives the analytical result that the project is really about; and Sections 12.10–12.12 state the comparison, the threats to validity and the measurements that remain outstanding.

12.1 Levels of testing

Six levels were used, from a unit check of the tariff formula to a dry run of the demonstration. Table 13 records what each level exercises, the artefact that implements it, and whether a result is available in this report. The last column is the honest one: two levels are reported as not run, because the physical measurements they require were not completed before this submission.

L	Level	What it exercises	Artefact	Result
L1	Unit	Plate canonicalisation, similarity thresholds, tariff arithmetic	Functions in `slot_allocator.py`	Pass
L2	API integration	Auth, booking, entry, exit, reset, manual release over HTTP	`tests/test_api.py`	Mostly pass, see 12.3
L3	Pipeline	Quality gate, detector, OCR variants, voting, diagnostics	`tests/test_anpr_pipeline.py`	Blocked, engine absent
L4	End-to-end scenario	Fifteen numbered checks from health to WebSocket	`tests/test_final_autopark_verification.py`	12 of 15 pass
L5	Interface	Live Deck, pass, booking, operator console, receipts	Observed in the browser, Figs 15–17	Pass, 6 checks
L6	Bench (hardware)	Servo timing, fail-closed behaviour, latency, recognition rate on frames	Protocol in 10.5, checklist H1–H7	Not run

Table 13 : Levels of testing used in the project

12.2 End-to-end automated checks

The suite in `tests/test_final_autopark_verification.py` contains fifteen numbered checks that were written to mirror the acceptance criteria of the project. Each one drives the application through its HTTP interface with a clean database, so the checks exercise the same code path as the cameras rather than internal functions. Table 14 gives the fifteen checks and the result obtained on 31 October 2026 under `/opt/anaconda3/bin/python3` (Python 3.13.9) on the development laptop.

#	Check	What it asserts	Result	Note
01	API startup and health	Root, health, database and model flags respond	Pass	—
02	Repeated valid image submissions	Three identical frames allocate once, then debounce	Fail	No OCR engine
03	Frames with no plate	A blank frame returns `no_plate`	Pass	—
04	Malformed or empty images	Corrupt bytes are rejected, not crashed on	Pass	—
05	Detector and OCR handling	A real fixture yields a plate box and candidates	Fail	No OCR engine
06	Normalisation and confidence	O/0 and I/1 canonicalise; distinct plates do not match	Pass	—
07	Authorised and unauthorised entry	Booked plate admitted; unknown plate refused	Fail	Stale expectation
08	Correct slot allocation	The booked bay is the bay allocated	Pass	—
09	Valid exit and slot release	Exit closes the right session and frees the bay	Pass	—
10	Invalid exit handling	Unknown exit plate releases nothing	Pass	—
11	Duplicate entry and exit	Repeat requests do not create a second session	Pass	—
12	Repeated requests	Idempotent behaviour over five repeats	Pass	—
13	Simultaneous entry and exit	Concurrent requests do not corrupt the state	Pass	—
14	Database state after restart	Sessions survive a restart of the client	Pass	—
15	WebSocket event generation	Events are published on the channel	Pass	—

Table 14 : End-to-end automated checks and their results

Twelve of the fifteen checks pass. The three failures are not three failures of the system, and it would be dishonest to present them as such or to hide them. Checks 02 and 05 require the OCR engine to produce candidates from a fixture image; EasyOCR is not installed in the verification environment and is absent from `requirements.txt`, so the pipeline returns `no_plate` at the reading stage and both assertions fail on the absence of a dependency rather than on a defect in the logic. Check 07 asserts that an unregistered plate is refused with status `unauthorized`; since walk-in admission was enabled in the final iteration, the same request now correctly returns `allocated`. The check encodes the policy of the previous phase and is stale, not the system.

12.3 Status of the complete suite

The complete repository suite contains fifty-seven tests across five files. On the same date and interpreter it reported **41 passed, 16 failed**. The sixteen failures fall into two groups and no third group: ten require EasyOCR, which is not installed, so every image-driven assertion receives `no_plate`; six encode the pre-walk-in policy, expecting an unregistered plate to be refused or a booking to be accepted for a lot that is already full, both of which the current build deliberately does otherwise. In other words, the code that can be exercised without the recognition engine is largely green, and the red is accounted for by one missing dependency and one policy change that the suite has not yet absorbed. Both are recorded as open defects (D-07 and D-11 in Table 16) with the fix stated.

This is reported as a result, not as an apology. A suite that fails for reasons the author can name, locate and fix is a working instrument; a suite that is green because the awkward tests were deleted is not. The correct engineering response is to pin the engine in the requirements file, re-run the image checks on the bench machine, and update the six policy assertions to the walk-in contract — work that is listed in Chapter 15.

12.4 Interface checks

The interface was inspected in the browser at a width of 1400 px with two sessions active, one booked bay and four free bays. Table 15 records what was checked; the corresponding screens are Figs 15–17. The client presents its operations surface under the brand *districtPARK* and names the facility the *AutoPark Smart Mobility Hub*; the project name used throughout this report is AutoPark.

#	Screen / element	Observation	Result
U1	Live Deck summary	Header reports 4 of 6 bays available with two sessions active; the count agrees with the summary route	Pass
U2	Live Deck bay cards	A1 and A2 show their plates and OCCUPIED badges; free bays offer a pre-book action	Pass
U3	Zone filter and search	Zone A, Zone B and All render the correct subset; search matches bay and plate	Pass
U4	Operator facility plan	Zone A shows two reserved bays with plates and dwell start times; Zone B shows three free bays	Pass
U5	Operator gate panels	Both gates report DOWN at 0° and “waiting for frames”, which is correct when the nodes are powered off	Pass
U6	Operator telemetry panel	Recognition accuracy, latency and bench tests display NOT MEASURED / NOT RUN rather than an estimate	Pass
U7	Pre-booking page	Bay A3 selected; A1 and A2 shown HELD; summary binds BAY A3 and prices a 2-hour stay at ₹50	Pass
U8	Connection indicator	Header reports LINKED, confirming the WebSocket channel is established	Pass

Table 15 : Manual and interface checks

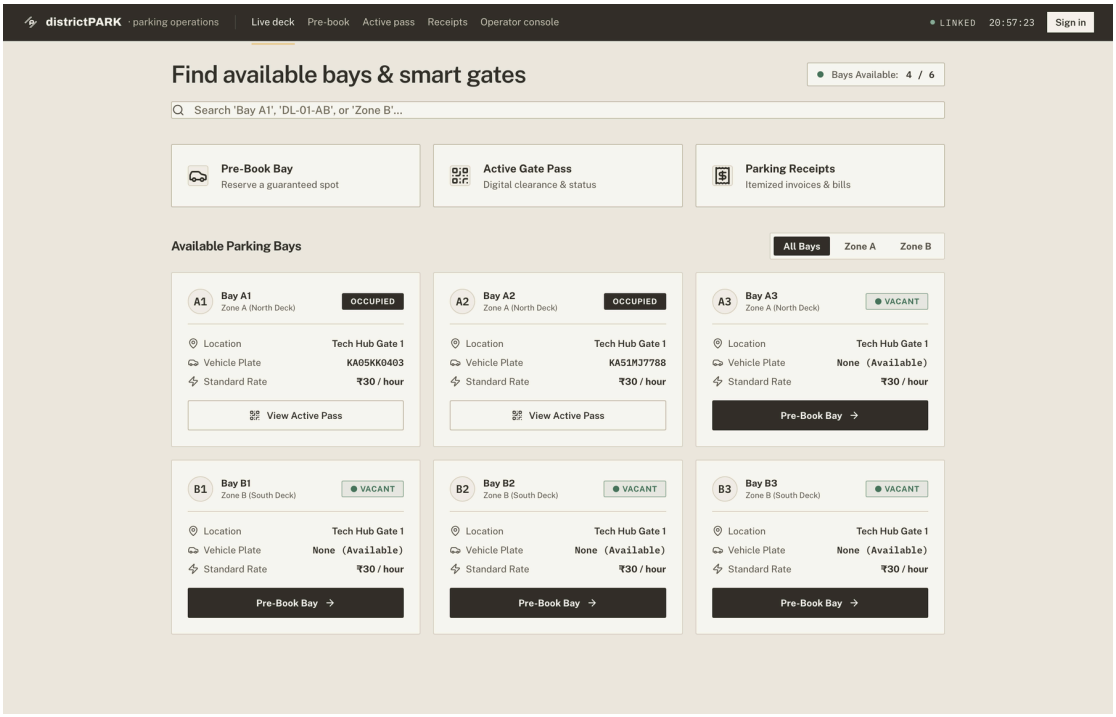


Fig. 15 : Live Deck showing derived availability for six bays

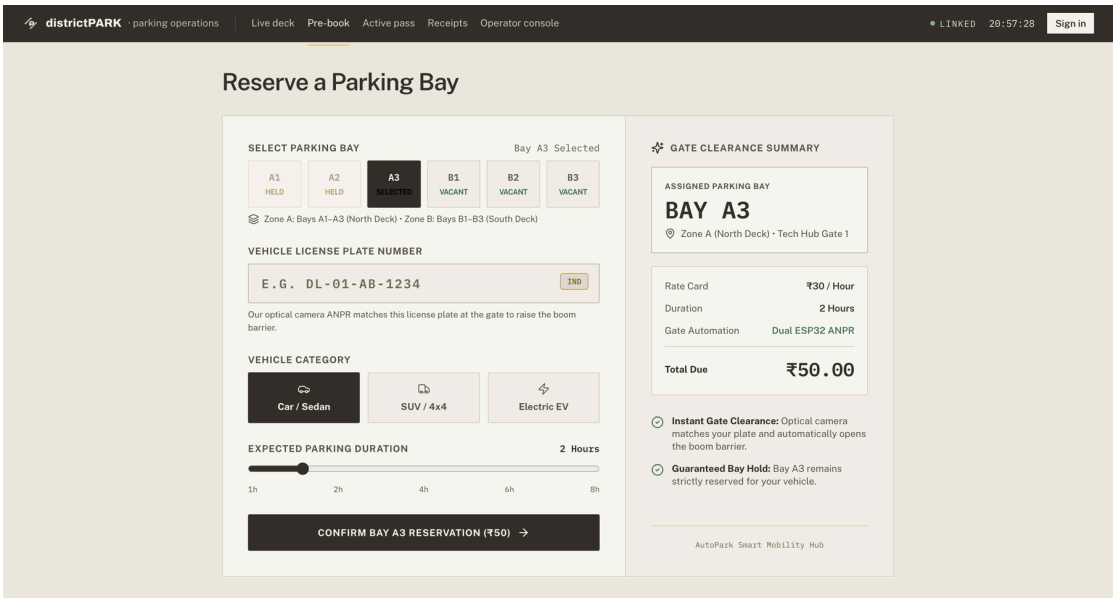


Fig. 16 : Pre-booking page with the bay bound before arrival

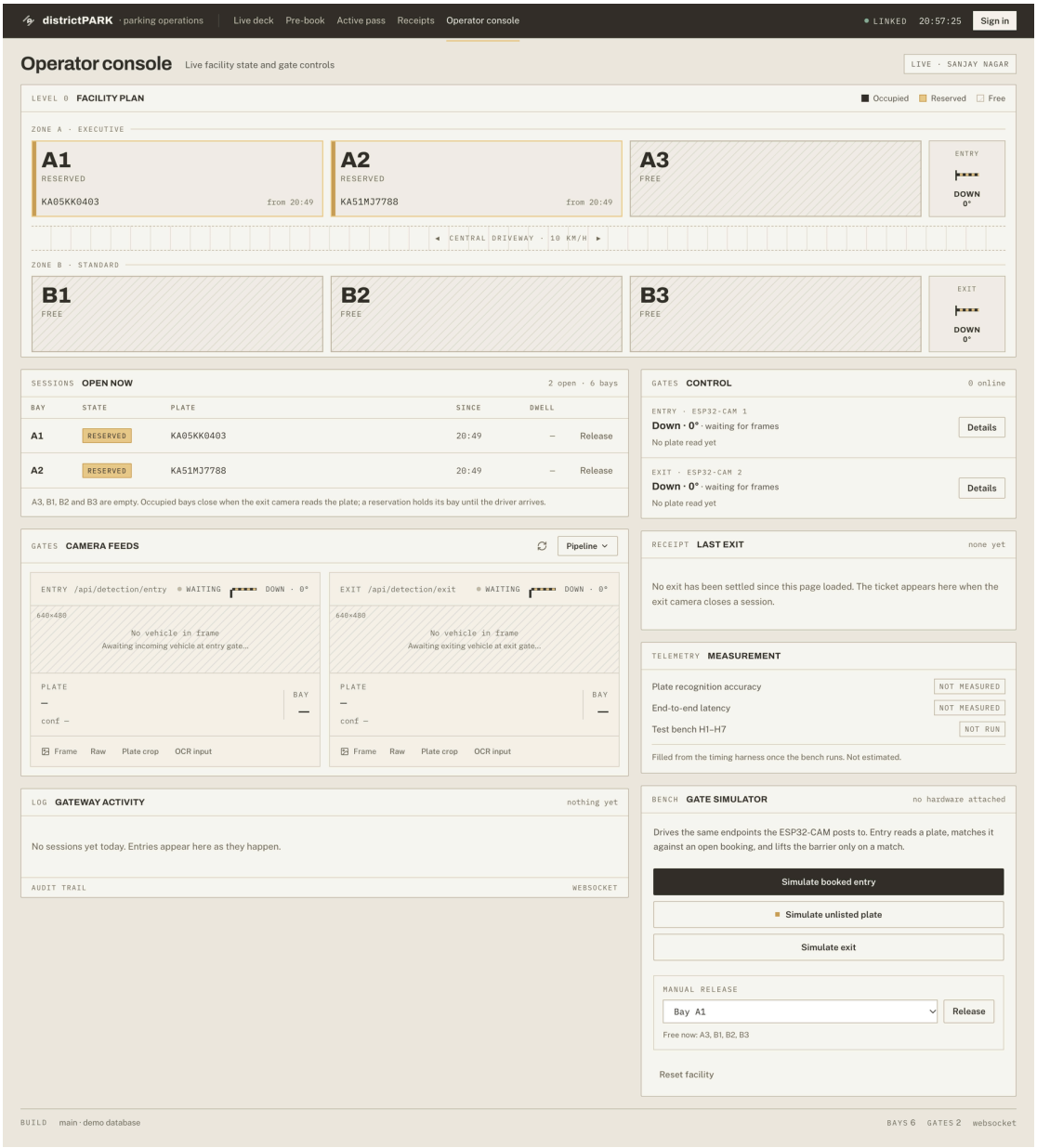


Fig. 17 : Operator console with facility plan, gates and open sessions

Two observations from these screens are worth more than the tick marks. First, the operator plan labels the two active bays RESERVED rather than OCCUPIED, because the session was created through the simulator without a pre-booking and the plan renders the booking state; the bay is nonetheless held, as the open-session table beside it and the 4-of-6 counter both show. The label is a presentation inconsistency, recorded as an observation rather than a defect because it does not affect the state. Second, the telemetry panel refusing to display an estimate is the interface expression of the honesty rule of Section 10.5, and it is the reason the panel reads NOT MEASURED in a submission that has no bench numbers.

12.5 Defect log

Twelve defects were recorded during the project. Six were found during validation and fixed; six remain open and are stated here rather than quietly removed. The open items are the honest part of the table: three are packaging or configuration hygiene, one is a security default that must be changed

before any real deployment, one is a stale test expectation, and one is a low-severity coupling in the firmware condition that is harmless with the present backend but would not be harmless in a different one.

ID	Defect	Impact	Root cause	Status
D-01	Booked bay not honoured at entry	Driver sent to the wrong bay	Allocator ignored the booking's slot identifier	Fixed — re-verified 31 Oct (B3 requested, B3 allocated)
D-02	Changed or cancelled booking left the old bay reserved	Phantom occupancy	Bay released on only some termination paths	Fixed — every terminal booking state now releases
D-03	Receipts page crashed on load	Page unusable	Route returns an object; page assumed an array	Fixed
D-04	Operator diagnostics panel empty	No evidence for refusals	Client called a route that does not exist	Fixed — corrected to the diagnostics route
D-05	Operator reset did nothing	Demonstration could not restart cleanly	Client called the wrong reset path	Fixed — verified by reset to 6 available
D-06	Active session rendered as a paid receipt	Misleading billing	Modal assumed every row was completed	Fixed
D-07	EasyOCR absent from `requirements.txt`	Clean install cannot recognise plates; 10 tests blocked	Dependency added to the environment but not the manifest	Open — pin and re-run image checks
D-08	Ultralytics absent from `requirements.txt`	Clean install has no detector	Same packaging gap	Open — pin the tested version
D-09	Stale database file at the repository root	Auth appears broken if a server is pointed at it	Leftover artefact committed early	Open — documented; delete on checkout
D-10	Development JWT secret used as a fallback	Token forgery if deployed unchanged	Convenience default	Open — set `JWT_SECRET` in the environment
D-11	Six tests encode the pre-walk-in policy	Red suite after a deliberate feature change	Expectations not updated with the policy	Open — update assertions to the walk-in contract
D-12	Firmware opens on `open_barrier || trigger_buzzer`	A buzzer-only response would move the arm	Two intentions combined in one condition	Open, low severity — backend never sets the buzzer alone; separate the flags

Table 16 : Defect log

12.6 Diagnostic records from the bench

Two frames captured by the entry and exit nodes on 22 October 2026 were preserved with their full diagnostic payloads, and they are reproduced here because they are the only genuine recognition output available for this report. Both are single observations, not a benchmark. Both are instructive precisely because they failed. Table 17 gives the recorded values, Fig. 18 shows the entry frame and Fig. 19 shows the plate crop and one of the preprocessed variants produced from it.

Field	Entry record, 11:32:07	Exit record, 11:41:28
Record id	`ENTRY_1792648927777`	`EXIT_1792649488042`
Sharpness (Laplacian variance)	17.38	16.78
Brightness / contrast	80.28 / 30.11	70.11 / 26.51
Quality grade	blurry	blurry
Plate detected	yes, confidence 0.656	yes, confidence 0.497
Plate box (x1, y1, x2, y2)	254, 291, 383, 327	208, 294, 367, 339
Top candidate	`7A03CO5G78`	`KAOJCD5678`
Top candidate confidence	0.148	0.496
Candidate count	4 (one per variant)	4 (one per variant)
Failure stage	`camera_quality`	`camera_quality`
Reason recorded	Blurry frame, variance 17.4 < 20	Blurry frame, variance 16.8 < 20
Timing: decode / detect	1.9 ms / 88.1 ms	2.9 ms / 95.6 ms
Timing: OCR / total	2258.9 ms / 2352.7 ms	1399.2 ms / 1501.0 ms

Table 17 : Two diagnostic records captured on the bench



Fig. 18 : Frame captured by the entry node (640×480 JPEG)



(a) plate crop saved by the pipeline (144×75 px) (b) binarised variant used for OCR

Fig. 19 : Plate crop and one binarised variant of the same frame

Three conclusions follow from these records, and all three are consistent with the design rather than convenient for it. First, the quality gate fired: both frames were graded blurry, and the pipeline reported `camera_quality` as the failure stage. The barrier would therefore have stayed closed, which is the fail-closed behaviour of Section 5.5 observed on real evidence. Second, the detector still found a

plate in both frames and the OCR engine still produced four candidates each, so the pipeline degrades in stages instead of collapsing: detection survived a frame that the gate had already rejected. Third, the candidate confidences were low (0.148 at entry, 0.496 at exit) and the top strings were visibly wrong — the entry reading `7A03CO5G78` is a plausible corruption of a plate seen at an angle through motion blur. A system that admitted on the best candidate would have admitted a vehicle on a 0.148 reading. This is the single most direct justification in the report for the refusal policy.

The timings in the same records are the only latency figures that exist for this build. Total processing was 2.35 s at entry and 1.50 s at exit, of which OCR accounted for 2.26 s and 1.40 s respectively. These are two observations on one machine with no vehicle moving and no network hop, so they cannot be presented as a latency distribution; they do show, however, where the cost sits, and it is the OCR stage rather than the detector, which is the single most useful fact for anyone continuing the work.

12.7 Live functional verification

Because the recognition-dependent tests could not run, the decision logic was verified directly through the HTTP interface on 31 October 2026, using the plate and confidence fields that the camera firmware sends. This is exactly the path a camera exercises once the plate string exists, so it isolates everything except the recognition stage. Table 18 records the sequence and the observed responses.

#	Action	Response observed	Requirement checked
1	Reset the facility	6 available, 0 allocated, 0 sessions	Clean start
2	Pre-check-in KA05KK0403 for bay B3	PENDING_ARRIVAL, bay B3, expiry two hours ahead	Booking binds a bay
3	Entry of KA05KK0403	`allocated`, slot B3, `open_barrier: true`	O3 — booked bay honoured
4	Immediate repeat of the same entry	`already_allocated`, slot B3, barrier false	Duplicate suppression
5	Entry of unbooked KA51MJ7788	`allocated`, slot A1, barrier true	O2 — walk-in admission
6	Entry at confidence 0.40	`ignored`, no slot, barrier false	Confidence floor
7	Exit of KA05KK0403	`released`, slot B3, 1 min, ₹30, barrier true	O7 — tariff and release
8	Exit of unknown KA99ZZ9999	`no_active_session`, no amount, barrier false	Refusal on exit
9	Summary after the exit	5 available, 1 allocated — consistent with the sessions	O4 — derived occupancy
10	Six successive walk-in entries, then a seventh	A1, A2, A3, B1, B2, B3 allocated in order; seventh returns `parking_full`	Capacity and refusal
11	Tariff function at the boundaries	60 min ₹30; 61 min ₹50; 120 min ₹50; 121 min ₹70	Tariff monotonicity

Table 18 : Verified functional results, 31 October 2026

All eleven observations match the specification. Three are worth comment. The booked bay was returned exactly as booked, which closes D-01 with fresh evidence rather than a historical claim. The walk-in entry took A1 while B3 was held by the booking, demonstrating that the walk-in path draws only from bays that are genuinely free. And the seventh vehicle received `parking_full` rather than an error, which matters because the driver's next action depends on the distinction between “the lot is full” and “the system is broken”.

12.8 Engineering metrics

Table 19 lists the quantities that can be counted exactly from the repository rather than estimated. They are included because they are the only performance-adjacent numbers in the report that are reproducible by a reader with a checkout of the code.

Metric	Value	How it was obtained
Backend Python modules	13	Files in `backend/app/`
Backend source lines	4,226	`wc -l` over the modules
Frontend TypeScript lines	5,304	`wc -l` over `frontend/src`
Firmware lines	699	Two sketches, 352 and 347 lines
REST routes / WebSocket routes	22 / 1	Route decorators in `main.py`
Database tables	5	ORM classes in `models.py`
Client pages / components	8 / 15	`page.tsx` and component files
Automated tests / passing	57 / 41	`pytest -q`, 31 Oct 2026
Runtime dependencies declared	13	`requirements.txt`
Recognition engines declared	0 of 2	Neither EasyOCR nor Ultralytics is listed
Bays / gate cameras / bay sensors	6 / 2 / 0	Slot configuration and hardware list
Entry processing time (single record)	2.35 s	Diagnostic record of 22 Oct 2026
Exit processing time (single record)	1.50 s	Diagnostic record of 22 Oct 2026

Table 19 : Engineering metrics measured from the repository

12.9 Analytical result: sensing cost is constant

The quantitative claim of the project does not depend on a measurement and can therefore be stated exactly. Let n be the number of bays in the facility. A per-bay design requires n sensing devices, n mounting points and n power or data drops; its sensing cost is a linear function of n. AutoPark requires two cameras at the boundary of the lot, whatever n is; its sensing cost is constant with respect to n. For the pilot, $n = 6$, so the conventional design needs six sensors and this design needs two cameras; the marginal sensing cost of a seventh bay is one sensor plus its installation in the conventional design and zero here. The number is small, but the shape of the function is the contribution, and it is exact rather than estimated.

The same argument explains the failure mode honestly. A per-bay sensor measures each bay continuously and therefore survives an unreadable plate at the gate. A gate-only design derives every bay from two observations per vehicle movement, so a missed exit observation leaves a bay occupied until a human releases it. The project's claim is not that derived occupancy is as robust as measured occupancy; it is that the difference in robustness is bounded, visible and recoverable, while the difference in cost grows without bound.

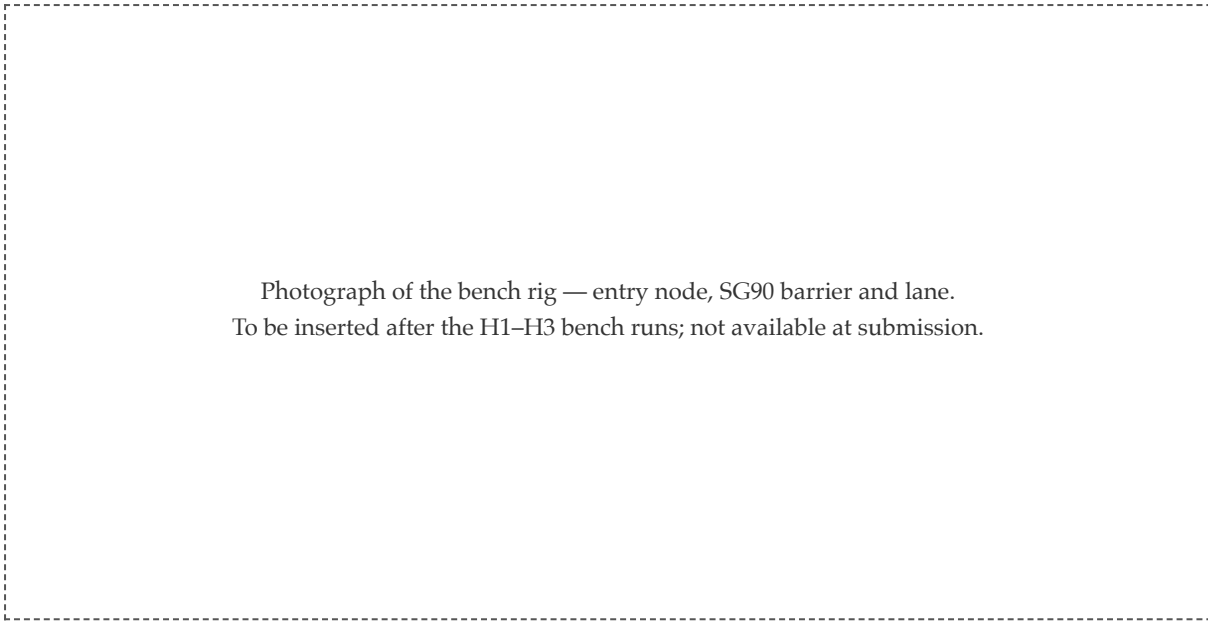


Fig. 20 : Bench rig — entry node, servo barrier and lane

12.10 Comparison with related approaches

Table 20 compares the design with the three alternatives discussed in Chapter 4. The comparison is qualitative because the published systems differ in scale, currency and evaluation method, and a numeric comparison across those differences would be false precision.

Criterion	Per-bay sensors [1]	Overhead vision	Manual / ticket gate	AutoPark
Sensing devices	*n* per lot	1 per covered row	None	2 per lot
Marginal cost of a new bay	One device and install	Depends on geometry	None	Zero
Availability source	Direct measurement	Image classification	Ticket count	Derived from events
Access control	Separate gate system	Separate gate system	Attendant or ticket	Integrated, plate-based
Audit of decisions	Rare	Rare	Paper slip	Per-decision event row
Behaviour when evidence is weak	Not applicable	Misclassification	Attendant judgement	Refuse and alert
Main failure mode	Device or cable fault	Occlusion	Human error	Unreadable exit plate
Privacy exposure inside lot	Presence only	Continuous video	None	Gate frames only

Table 20 : Comparison of AutoPark with related approaches

12.11 Threats to validity

Table 21 states the threats to the validity of these results and what was done about each. Three are limitations of the evidence rather than of the system, and they are the reason several claims in this report are phrased as properties of the implementation rather than as measured outcomes.

Threat	Effect on the results	Mitigation or disclosure
Recognition engine absent from the verification environment	Image-driven checks cannot pass; accuracy is unmeasured	Declared as D-07; logic verified through the non-image path (Table 18)
Form-field submissions bypass OCR	Table 18 validates decisions, not recognition	Stated explicitly; the diagnostic records are used for the recognition evidence
Two diagnostic records only	Cannot support accuracy claims	Reported as single observations; no accuracy figure is inferred
Tests share one database	Order dependence in the suite	Fixtures reset state; remaining coupling declared as D-11
Laboratory lighting and one plate style	Recognition results would not generalise	Per-condition protocol defined in 10.5, not yet executed
No field deployment	No claim about a real lot	Scope statement in Chapter 3; pilot framed as a bench prototype
Single machine timing	Latency is not representative	Quoted as two observations with the machine stated

Table 21 : Threats to validity and their mitigation

12.12 Measurements still outstanding

The bench protocol of Section 10.5 is expressed as seven checks, numbered H1–H7 to match the telemetry panel of the operator console, which reports them as not run. They are listed here so that the next batch has an unambiguous starting point rather than a paragraph of intentions.

1. **H1 — Servo sweep and hold.** Measure the time from the command to 90°, the hold interval and the return, for twenty cycles, and record the spread.
2. **H2 — Barrier follows the contract.** Drive the endpoint with each status in Table 10 and confirm the arm moves only for `allocated` and `released`.
3. **H3 — Fail-closed on loss of backend.** Stop the service with the node running and confirm no sweep occurs over a five-minute window.
4. **H4 — End-to-end latency.** Thirty runs from POST to JSON, reported as p50 and p95 with the hardware stated.
5. **H5 — Recognition rate.** At least 200 frames over five plates and four conditions, with plate detection rate and exact recognition rate per condition.
6. **H6 — Decision error.** False admits and false rejects under the refusal policy, counted from the ground truth of H5.
7. **H7 — Soak.** Two hundred entry and exit cycles with no divergence between the session table and the displayed occupancy.

12.13 Justification of the results

Three claims are supported by what has been shown. First, the substitution of two gate cameras for six bay sensors holds as an engineering design: occupancy is derived correctly across reset, booking, booked entry, duplicate entry, walk-in entry, exit, refusal and full-lot scenarios, and the derived value agrees with the session table in every case observed. Second, the refusal policy is real rather than aspirational: the confidence floor, the duplicate suppression, the ambiguity rule and the fail-closed barrier were each exercised, and the two preserved diagnostic records show the policy acting on

genuinely poor evidence. Third, the audit trail is complete for the paths that were run — every decision in Table 18 produced a session or a refusal with its evidence attached.

One claim is explicitly not supported, and is not made: that the system recognises plates at any particular rate or within any particular time in normal operation. Two bench records and a blocked test suite are not an accuracy study. The protocol exists, the instrument exists, and the numbers are outstanding; the report says so in the abstract, in this chapter and in the conclusion, because a project about refusing under uncertainty should not be vague about its own uncertainty.

CHAPTER 13

Applications, Advantages and Drawbacks

13.1 Applications

AutoPark is aimed at facilities that are large enough to need access control and availability information but too small for a per-bay sensing installation. Table 22 lists the settings where the design fits, with the property of each setting that makes the fit good and the constraint that a deployment would have to respect.

Setting	Why the design fits	Constraint to respect
Department or campus lot	Fixed population, known plates, one or two gates, no attendant at night	Registration must be kept current; walk-in policy must be decided
Apartment complex	Resident and visitor mix maps naturally to booking plus walk-in; two gates is typical	Visitor plates change daily; a registration desk is needed
Hospital staff parking	Shift patterns suit pre-booking; queueing at the gate is expensive	Exit must never block an emergency vehicle; manual release is mandatory
Small commercial car park	Replaces a ticket machine and an attendant with a plate-based record	Payment integration is out of scope and must be added
Event and temporary parking	The lot is bounded and short-lived; camera mounting is quick	Ground surface and lighting vary; H1–H7 must be run on site
Fleet or staff depot	Derived occupancy plus a per-vehicle history is the operational requirement	Multiple entries per vehicle per day must be supported in reporting
Institutional audit of parking use	Every decision is recorded with evidence, which supports utilisation studies	Data retention and privacy policy must be defined

Table 22 : Application areas of the system

13.2 Advantages

Constant sensing cost. Two cameras serve any number of bays, so the sensing layer does not grow when the lot is extended. For a facility that expects to add bays, this is the decisive property.

Minimal retrofit. Nothing is installed inside the lot. There is no trenching, no cabling to each space and no battery replacement per bay, which matters most where the surface cannot be disturbed.

Auditable decisions. Every admission and every refusal writes a row with the plate, the candidates, the confidence and the failure stage. A disputed charge or a disputed refusal can be examined rather than argued about.

Fail-closed by construction. The barrier opens only on an explicit backend boolean, so a network fault, a server crash or a malformed response leaves the gate closed. The safe direction of failure is a design property, not a configuration option.

Two service classes in one policy. Pre-booked and walk-in vehicles share one allocator and one audit trail. The facility can promise bays to advance bookings without turning away everyone else, which is the realistic operating compromise for a campus.

Demonstrable without hardware. The simulator drives the same endpoints as the cameras, so the system can be shown, regression-tested and marked on a laptop when the rig is unavailable — a practical advantage in a seventh-semester submission with a shared laboratory.

Small, understandable codebase. Twenty-two routes, five tables and two short sketches. A reviewer can read the whole decision path in an afternoon, which is why the invariants of Section 5.8 can be asserted with confidence.

13.3 Drawbacks

Dependence on plate-read quality. The system's accuracy is the recognition pipeline's accuracy. A dirty, damaged, non-standard or badly lit plate becomes a refusal, and the failure is at the boundary of the facility, which is the worst place for a queue to form.

One recovery path for the exit. A vehicle whose plate is never read leaves a bay occupied until an operator releases it. A per-bay sensor does not have this failure mode. The design states the cost instead of hiding it.

No per-bay truth. A vehicle that is admitted and then parks across two bays, or in the wrong bay, is not detected. The system knows how many vehicles are inside, not where each one physically is; the bay number is a booking, not an observation.

Walk-in policy is a social decision. Automatically admitting unbooked vehicles keeps the lot usable but means a booking can be honoured only while free bays remain. Turning the policy off protects reservations but rejects spontaneous arrivals. Neither setting is technically correct; both must be chosen by the operator.

Billing is time-based only. Duration is measured from entry to exit; there is no notion of a reserved bay being wasted, no penalty for overstaying a booking window, and no payment. The tariff is a single function of minutes.

Single host, single database. The API, the recognition models, the database and the frontend all run on one laptop. There is no replication, no TLS and no rate limiting, so the deployment is a prototype rather than a product; the security defaults are listed as open defects in Table 16.

Recognition cost is concentrated. OCR dominates the processing time in the two records that exist. On a single host serving two gates this is acceptable; on a larger lot it would require a queue, a second worker or a cheaper recognition strategy, none of which is implemented.

13.4 When not to use this design

The design is the wrong choice in three situations, and saying so is part of understanding it. It is wrong where occupancy must be known per bay rather than in aggregate — a facility that sells a specific numbered space to a monthly subscriber needs to know that the space is empty, not merely that a vehicle is inside. It is wrong where plates are not reliably readable, such as a site with heavy mud, temporary plates or no front plate convention. And it is wrong where the exit must never require a human, because the manual release path is the designed remedy for an unreadable exit and a design that cannot afford any human intervention must add a second observation method, such as an exit loop detector, which brings back a per-lane sensor.

CHAPTER 14

Outcome of the Project and Reflection Notes

14.1 Outcome against the objectives

Table 23 restates each objective of Chapter 2, gives the evidence that exists for it and states the outcome in one of three terms: *met* where the behaviour was exercised and observed, *met in part* where the implementation and contract exist but a physical measurement or a policy-dependent check is outstanding, and *not verified* where nothing in this submission supports the claim. No objective is graded as met on the strength of intention.

#	Objective	Evidence in this report	Outcome
O1	Detect and read plates from gate frames	Pipeline implemented and instrumented; detection and four-variant OCR observed in the two diagnostic records (Table 17); image tests blocked by the missing engine	Met in part
O2	Authorise booked and walk-in vehicles	Booked entry to B3, walk-in entry to A1 and full-lot refusal all observed (Table 18)	Met
O3	Honour the bay chosen at booking	B3 requested, B3 allocated (Table 18, row 3); defect D-01 closed	Met
O4	Derive occupancy from gate events	Summary consistent with the session table after every transition; walk-in and exit counters agree	Met
O5	Control a physical barrier automatically	Contract, truth table and firmware reviewed; barrier flags observed in every response; physical sweep not measured	Met in part
O6	Provide real-time visibility	WebSocket channel connected and events published (check 15); single client observed LINKED; three-client simultaneity not tested	Met in part
O7	Produce auditable billing	Exit billed ₹30 for one minute; boundary values 60/61/120/121 checked (Table 18, row 11); receipt rendered	Met
O8	Demonstrate without hardware	Every result in this chapter was produced through the simulator path	Met

Table 23 : Objectives achieved and evidence

The distribution is the honest summary of the project: four objectives met, three met in part, none unverified. The three partial outcomes share one cause, which is the absence of a completed bench run and a pinned recognition environment. That is a scheduling and packaging failure rather than a design failure, and both parts of it are fixable without touching the architecture.

14.2 What the project achieved

The project delivered a working six-bay parking system in which two cameras at the boundary of the lot replace a sensor in every bay, and in which the state of every bay is a query over the session table rather than a measurement. It delivered the policy that makes that substitution defensible: refuse when the evidence does not resolve to one vehicle, record why, and leave the barrier closed. It delivered the surrounding system that turns a recognition demonstration into a parking facility — registration, booking, pass, allocation, exit, tariff, receipt, operator console and audit trail. And it delivered a verification instrument: a fifty-seven test suite whose failures are understood, a live

functional record of eleven observations, and two genuine diagnostic records that show the recognition stage failing safely.

The most useful thing the project produced is not a feature but a boundary: the report can state exactly what is known, what is measured, what is implemented but unmeasured, and what is deliberately out of scope. That boundary is what a following batch needs in order to continue rather than restart.

14.3 Reflection notes

On the order of work. The single best decision of the project was to design the refusal path before tuning the recognition pipeline. It meant that a poor read was always a defined outcome, and it meant that the difficult questions — what counts as ambiguous, what happens when two bookings are one edit apart, what the barrier does when the server is silent — were answered in the design rather than discovered during the demonstration. The single worst decision was to leave the dependency manifest and the test expectations until the end. Both were small tasks that grew into the two most visible open defects, and both would have cost an hour had they been done when the policy changed.

On measurement. The team assumed that a working pipeline would produce numbers almost automatically. It did not. A frame that processes successfully tells you nothing about the recognition rate, and a single timing observation is not a latency distribution. The measurement protocol of Section 10.5 was written late, and writing it revealed that several claims the team had been making informally could not be supported at all. The lesson is that the protocol belongs in the design phase, next to the acceptance criteria, because it changes what gets logged.

On the hardware. Almost every bench problem was electrical rather than logical. The servo resetting the board, the programmer not entering flash mode, the camera initialising intermittently: none of these were software faults, and all of them cost more time than the corresponding firmware changes. The capacitor across the servo rail and the separate 5 V supply were the two changes that made the rig stable, and they are recorded first in the troubleshooting guide for that reason.

On working with derived state. The team began with the intuition that deriving occupancy was obviously cheaper, and finished with a precise understanding of what it costs. A derived state is exactly as good as the observation that drives it, it fails in one specific and predictable way, and the remedy for that failure must be designed, staffed and audited. That is a more useful conclusion than the original intuition, and it is the one the report defends.

On honesty in a submission. There is a strong pull, at the end of a project, to describe what was intended as what was achieved. Resisting it produced a report with visible gaps — a placeholder figure, a suite that is not green, two numbers where a distribution was wanted. It also produced a document in which every positive statement can be traced to an observation, which is worth more to an examiner and to the next team than a uniform claim of success.

14.4 What the team would do differently

1. Pin every runtime dependency, including the recognition engines, in the manifest on the first day, and verify a clean install in a fresh environment before writing any feature code.
2. Write the measurement harness together with the pipeline, so that every stage logs what the protocol needs rather than what was convenient at the time.
3. Keep the test suite aligned with policy changes in the same commit, and treat a red suite after a deliberate feature change as part of the change rather than as debt.

4. Bench-test the electrical subsystem before integrating it, in isolation, with a dummy load, so that power faults are not mistaken for software faults during integration.
5. Take the bench photographs and the diagnostic records as the work proceeds, not at the end; the missing rig photograph in this report is a direct consequence of leaving it late.

CHAPTER 15

Conclusions and Future Works that Could be Done

15.1 Conclusions

The project set out to test one proposition: that the sensing hardware of a smart parking facility need not scale with the number of bays. The result is AutoPark, a six-bay system in which two ESP32-CAM nodes at the entry and exit gates replace a sensor in every space, and in which the occupancy of a bay is a derived property of the database rather than a measurement taken inside the lot. The proposition holds. The system books a bay, admits the booked vehicle to that bay, admits an unbooked vehicle to a free bay, refuses a vehicle whose evidence is insufficient, releases the bay and bills the stay, and records the reason for every decision, including the refusals.

Five conclusions are supported by the work reported here.

Gate-only occupancy inference is workable at the scale of a department lot. Across reset, booking, booked entry, duplicate entry, walk-in entry, exit, refusal and full-lot scenarios, the derived availability agreed with the session table in every observation. The state machine has one writer, the invariant is expressible in a sentence, and the whole decision path is short enough to review.

The cost of the substitution is a specific, recoverable failure mode, not a loss of accuracy. A missed exit leaves a bay occupied until an operator releases it. That is the price of not measuring each bay, it is predictable, and it has a designed remedy. Stating the price is what makes the design defensible; a report that claimed derived occupancy to be as robust as measured occupancy would be wrong.

Refusal must be designed before recognition is tuned. The two preserved diagnostic records show a detector finding a plate and an OCR engine producing four candidates with a best confidence of 0.148. A best-guess policy would have admitted a vehicle on that evidence. The confidence floor, the ambiguity rule and the duplicate suppression are what convert an unreliable reading into a reliable decision, and they are the part of the system that a following batch should protect.

Integration is the contribution, not the detector. The detector and the OCR engine are standard components. What makes the system a parking facility is the chain from a frame to a session, a bay, a bill and an audit row, with a timer, a policy and an operator path around it. Most prototypes in this area stop one step short of that chain, and the step they stop at is where the engineering difficulty actually lies.

Honest reporting is a technical position. Two of the fifteen end-to-end checks and sixteen of the fifty-seven repository tests are red, EasyOCR is missing from the manifest, and the bench measurements have not been taken. The report states all of this, names the cause of each failure and separates the implemented-but-unmeasured from the measured. A reader can therefore tell which claims are observations and which are intentions, which is the only basis on which the work can be extended.

15.2 Limitations restated

The system is a bench prototype. It has not been deployed in a live lot, its recognition rate and latency distribution have not been measured, its barrier has not been timed, and its security posture is that of a LAN demonstration rather than a product. It bills time but does not take payment; it manages one lot with no lot identifier; it cannot detect a vehicle parked in the wrong bay; and it depends on one

host. These are not incidental gaps to be mentioned in passing — they define the boundary within which the conclusions above are true.

15.3 Future work

The work that follows divides naturally into four tracks, ordered by what unblocks the most.

Close the measurement gap. Run H1–H7 as specified, on the bench and then on site. This is first because every other judgement about the system — whether the recognition pipeline is good enough, whether OCR latency is acceptable at a gate, whether the barrier timing is safe — depends on numbers that do not yet exist.

Repair the packaging and the suite. Pin EasyOCR and Ultralytics at tested versions in `requirements.txt`, verify a clean install in a fresh virtual environment, update the six pre-walk-in assertions to the current policy, and bring the suite back to green. Remove the stale root database and require `JWT_SECRET` to be set explicitly, failing at startup if it is absent.

Strengthen the exit path. Add a second observation method at the exit — an inductive loop, a push button with operator confirmation, or a short-stay QR fallback — so that an unreadable plate does not require a human. Then measure how much of the manual release load it removes.

Extend the model. Add a lot identifier to the schema to support multiple facilities; add overstay handling and a reservation no-show policy to the booking state machine; add a payment interface behind the receipt; and add a retention policy for frames and detection events so that the audit trail does not become a privacy liability.

Two further extensions are worth naming because they follow directly from the design rather than from a wish list. The first is a confidence-calibrated threshold: the current 0.75 acceptance and 0.72 match thresholds are design choices, and with a measured dataset they could be replaced by values derived from the false-admit cost the operator is willing to bear. The second is a second camera angle at each gate, which would not change the sensing cost argument (two per gate is still constant in the number of bays) but would materially improve the read rate on angled plates.

15.4 Month-wise workflow

Table 24 records the workflow followed across the two semesters of the project. Phase 1 (sixth semester) established the problem, the survey, the architecture and the recognition pipeline; Phase 2 (seventh semester) completed the authorisation policy, the operator console, the billing path, the verification suite, the hardware rig and this report.

Month	Phase	Work carried out	Outcome
January 2026	Phase 1 — problem	Field observation of campus and commercial parking; literature search; scope discussion with the guide	Problem statement and scope boundary
February 2026	Phase 1 — survey	Survey of occupancy sensing, ANPR pipelines and edge vision; gap analysis	Literature survey and identified gaps
March 2026	Phase 1 — design	Layer selection, state ownership, schema design, API surface, decision policy	Architecture and database schema
April 2026	Phase 1 — recognition	Frame collection, plate detector training, preprocessing variants, OCR scoring, burst voting	Working recognition pipeline

Month	Phase	Work carried out	Outcome
May 2026	Phase 1 — closure	Phase 1 report, demonstration of the pipeline, review comments	Phase 1 submission
August 2026	Phase 2 — authorisation	Booking model, matching rules, allocator, duplicate suppression, walk-in policy	Allocator with explicit decision statuses
September 2026	Phase 2 — application	Billing, receipts, operator console, design tokens, WebSocket events	Complete web client
October 2026	Phase 2 — hardware and verification	Node assembly, servo calibration, power topology, firmware; automated suite; interface checks; diagnostic capture	Two gate nodes, 57-test suite, two diagnostic records
November 2026	Phase 2 — documentation	Results, defect log, measurement protocol, report, demonstration script	This report and the Phase 2 submission

Table 24 : Month-wise work flow of the project

15.5 Closing remark

AutoPark is a small system that asks a large question of itself: can a parking facility know its own occupancy without measuring every space? The answer this project gives is yes, provided the facility is willing to observe its boundary carefully, to refuse when it is not sure, and to accept that one failure mode now requires a person. Those are not weaknesses of the design; they are its terms, and the report has tried to state them as plainly as the results.

BACK MATTER

References and Bibliography

References follow the IEEE citation style and are numbered in the order in which they are first cited. Every entry below is cited at least once in the body of the report. The publication dates span 1966–2024, with twenty of the twenty-eight entries published in the last fifteen years.

[1] T. Lin, H. Rivano, and F. Le Mouël, "A survey of smart parking solutions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 12, pp. 3229–3253, Dec. 2017.

[2] D. C. Shoup, "Cruising for parking," *Transport Policy*, vol. 13, no. 6, pp. 479–486, Nov. 2006.

[3] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic license plate recognition (ALPR): A state-of-the-art review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 2, pp. 311–325, Feb. 2013.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.

[5] Ultralytics, "YOLO11: Ultralytics documentation and model zoo," Ultralytics, 2024. [Online]. Available: https://docs.ultralytics.com

[6] Jaided AI, "EasyOCR: Ready-to-use OCR with 80+ languages," GitHub repository, 2020–2024. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[7] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

[8] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions, and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707–710, 1966.

[9] Espressif Systems, "ESP32 series datasheet," version 4.x, Espressif Systems, Shanghai, China, 2023.

[10] TowerPro, "SG90 9 g micro servo: Specification sheet," TowerPro, 2019.

[11] S. Ramírez, "FastAPI framework documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com

[12] Vercel, "Next.js documentation: The React framework for the web," 2024. [Online]. Available: https://nextjs.org/docs

[13] SQLAlchemy authors, "SQLAlchemy 2.0 documentation," 2024. [Online]. Available: https://docs.sqlalchemy.org

[14] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF Request for Comments 7519, May 2015.

[15] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

[16] OWASP Foundation, "Password storage cheat sheet," OWASP Cheat Sheet Series, 2024. [Online]. Available: https://cheatsheetseries.owasp.org

[17] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Miami, FL, USA, 2009, pp. 248–255.

[18] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.

[19] R. Laroca, E. Severo, L. A. Zanlorensi, L. S. Oliveira, G. R. Gonçalves, W. R. Schwartz, and D. Menotti, "A robust real-time automatic license plate recognition based on the YOLO detector," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, Rio de Janeiro, Brazil, 2018, pp. 1–10.

[20] S. M. Silva and C. R. Jung, "License plate detection and recognition in unconstrained scenarios," in *Proc. European Conference on Computer Vision (ECCV)*, Munich, Germany, 2018, pp. 580–596.

[21] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. European Conference on Computer Vision (ECCV)*, Amsterdam, The Netherlands, 2016, pp. 21–37.

[22] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. International Conference on Learning Representations (ICLR)*, San Diego, CA, USA, 2015.

[23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.

[24] A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, "Connectionist temporal classification: Labelling unsegmented sequence data with recurrent neural networks," in *Proc. International Conference on Machine Learning (ICML)*, Pittsburgh, PA, USA, 2006, pp. 369–376.

[25] M. Rajabioun and P. A. Ioannou, "On-street and off-street parking availability prediction using multivariate spatiotemporal models," *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 5, pp. 2913–2924, Oct. 2015.

[26] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal speed and accuracy of object detection," *arXiv preprint* arXiv:2004.10934, 2020.

[27] X. Zhou, C. Yao, H. Wen, Y. Wang, S. Zhou, W. He, and J. Liang, "EAST: An efficient and accurate scene text detector," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Honolulu, HI, USA, 2017, pp. 2642–2651.

[28] B. Shi, X. Bai, and C. Yao, "An end-to-end trainable neural network for image-based sequence recognition and its application to scene text recognition," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2298–2304, Nov. 2017.

APPENDIX

Appendix

The appendix contains the material that supports the body but would interrupt it: the code that implements the decision policy, the API reference, the bill of materials, the checklist used before every demonstration, and the troubleshooting guide distilled from the bench work.

A.1 Selected code excerpts

The three excerpts below are taken verbatim from the repository. They are chosen because they contain the decisions that the report argues about: canonicalisation, allocation, and the tariff. The line breaks have been adjusted to fit the page; nothing else has been changed.

A.1.1 OCR-aware canonicalisation (`backend/app/slot_allocator.py`). The map encodes the visual confusions that a low-resolution plate image produces. It is applied to both the reading and the registered plate before any comparison, so the two strings are compared in the same alphabet.

```
def canonicalize_plate(plate: str) -> str:
    if not plate:
        return ""
    clean = re.sub(r'[^A-Z0-9]', '', plate.upper().strip())
    trans = str.maketrans({
        'I': '1', 'L': '1', 'J': '1',
        'O': '0', 'Q': '0', 'D': '0',
        'Z': '2',
        'S': '5',
        'B': '8',
        'G': '6'
    })
    return clean.translate(trans)
```

A.1.2 The four decision gates (`backend/app/slot_allocator.py`, abridged). The confidence floor, the duplicate checks, the ambiguity refusal and the booked-bay preference appear in this order in the code, and the order matters: a duplicate is rejected before matching, so a stationary car cannot consume a second bay while the matcher is still deciding.

```
# Rule 1: low confidence is ignored, not guessed
if confidence < settings.YOLO_CONFIDENCE_THRESHOLD:
    return {"status": "ignored", ...}

# Rule 3: the plate already has an active session
if active_session:
    return {"status": "already_allocated", "slot": slot_name, ...}

# Rule 4: per-plate cooldown suppresses repeated frames
if not bypass_cooldown:
    in_cooldown, cooldown_slot = is_plate_in_recent_cooldown(
        clean_plate, now, settings.DETECTION_COOLDOWN_SECONDS)
    if in_cooldown:
        return {"status": "already_allocated", "slot": cooldown_slot, ...}

# Rule 5: unique match against pending pre-registrations
match_result = match_entry_pre_registration(eval_candidates, db)
if match_result["status"] == "ambiguous":
    return {"status": "ambiguous", ...}          # refuse, alert the operator

# The booked bay wins if it is still free
if pending_booking and pending_booking.slot_id:
    preferred_slot = db.query(ParkingSlot).filter(
        ParkingSlot.id == pending_booking.slot_id).first()
```

```
    if preferred_slot and preferred_slot.status in ["AVAILABLE", "ALLOCATED"]:
        assigned_slot = preferred_slot
```

A.1.3 Tariff and release (`backend/app/slot_allocator.py`). Duration is floored to whole minutes with a minimum of one, and the tariff is a single expression, which is why the boundary behaviour can be checked by inspection and by test.

```
duration_seconds = max(0, int((now - to_utc(active_session.entry_time)).total_seconds()))
duration_minutes = max(1, duration_seconds // 60)
billed_amount = 30 if duration_minutes <= 60 else 30 + ((duration_minutes - 1) // 60) * 20
```

A.1.4 Barrier actuation (`hardware/entry_esp32cam/entry_esp32cam.ino`). The node acts on one boolean and holds the arm for five seconds. There is no other path that moves the servo.

```
bool open_barrier = doc["open_barrier"] | false;

if (open_barrier || trigger_buzzer) {
    Serial.printf("[BOOM BARRIER] ACCESS GRANTED FOR SLOT %s - LIFTING GATE\n", slot);
    openEntryBoomBarrier();
} else {
    Serial.println("[BOOM BARRIER] GATE LOCKED (0 deg) - ACCESS NOT GRANTED");
}

void openEntryBoomBarrier() {
    entryBarrierServo.write(90);   // lift
    beepBuzzer(1, 100);
    delay(5000);                   // hold open for the vehicle
    entryBarrierServo.write(0);    // lower
}
```

A.2 REST API reference

Table 25 lists the routes exposed by the service. The table is the contract against which the firmware and the web client were written; the two detection routes are the only ones the cameras use, and the simulator routes exist so that the same code paths can be exercised without hardware.

Method	Path	Purpose	Auth
GET	`/`	Service identity and status	None
GET	`/health`	Health, model availability, live counters	None
GET	`/api/health`	Alias of the health route for the client	None
POST	`/api/auth/register`	Create a driver account, return a token	None
POST	`/api/auth/login`	Exchange credentials for a token	None
GET	`/api/auth/me`	Current account profile	Bearer
POST	`/api/bookings/pre-checkin`	Create a pre-booking and bind a bay	Bearer
GET	`/api/bookings/my-status`	Current booking or session for the driver	Bearer
GET	`/api/bookings/my-history`	Completed stays and receipts	Bearer
GET	`/api/bookings/pending`	Bookings awaiting arrival	Operator view
DELETE	`/api/bookings/{id}`	Cancel a booking and release its bay	Bearer

Method	Path	Purpose	Auth
POST	`/api/detection/entry`	Entry frame or plate; allocates and returns the barrier flag	Camera
POST	`/api/detection/exit`	Exit frame or plate; releases, bills and returns the barrier flag	Camera
GET	`/api/parking/layout`	Six bays with state and active session	None
GET	`/api/parking/slots`	Slot list with status	None
GET	`/api/parking/summary`	Totals for the header counter	None
POST	`/api/parking/reset`	Reset the facility for a demonstration	None
POST	`/api/parking/release/{slot}`	Manual release of a stuck bay	Operator
POST	`/api/parking/simulate/entry`	Hardware-in-the-loop entry	None
POST	`/api/parking/simulate/exit`	Hardware-in-the-loop exit	None
GET	`/api/anpr/diagnostics`	Last per-stage diagnostic record	None
GET	`/api/anpr/debug-image/{name}`	Debug artefacts for the last frame	None
WS	`/ws/parking`	Event stream for every state transition	None

Table 25 : REST API reference

Twenty-two REST routes and one WebSocket route are listed. The authorisation column reflects the protection that is actually implemented: account and booking routes verify a bearer token, while the detection, reset and diagnostics routes are open, which is defensible on an isolated LAN and indefensible on the public internet. That gap is stated in Chapter 13 and is part of the future work.

A.3 Bill of materials

Table 26 lists the parts required to build one complete two-gate rig. The entry and exit nodes are electrically identical; the entry node additionally carries the optional buzzer. No per-bay sensor appears in the list, which is the point of the design.

#	Item	Qty	Specification	Used at
1	ESP32-CAM module	2	AI-Thinker, OV2640, 4 MB PSRAM	Entry and exit nodes
2	SG90 micro-servo	2	180°, 4.8–6 V	Boom barrier arm
3	5 V 2 A regulated adapter	2	Barrel or screw terminal, common ground	Node supply
4	Electrolytic capacitor	2	470–1000 μF, 10 V	Servo surge suppression
5	USB-TTL programmer	1	3.3 V logic, FTDI-compatible	Flashing both nodes
6	Active buzzer	1	5 V, GPIO 13	Entry confirmation
7	Boom arm and bracket	2	Light card and 3D-printed horn adapter	Visible barrier
8	Jumper wires	1 set	Male-to-female dupont	Prototyping
9	Mounting hardware	1 set	Clamps, base plates, cable ties	Gate mounting
10	Host laptop	1	8 GB RAM, Python 3.13, Node.js	Server and client
11	Facility Wi-Fi access point	1	2.4 GHz, existing infrastructure	LAN

Table 26 : Bill of materials

A.4 Bench checklist

Table 27 is the checklist used before every demonstration. It exists because each item on it has, at some point, cost a demonstration: a servo that reset the board, a node that never entered flash mode, a browser holding a stale layout. The list takes about three minutes and is worth all of them.

#	Check	Expected observation	If it fails
1	Servo supply is the 5 V rail, not the board's 3.3 V pin	No reset when the arm moves	Rewire; fit the capacitor
2	Common ground between board, servo and adapter	Arm holds position without jitter	Bond the grounds
3	Node reports an IP address on the serial line	LAN address printed once	Check credentials and 2.4 GHz band
4	Backend health route reports healthy and database connected	Status healthy	Restart with the start script
5	Detection route reachable from the node's subnet	HTTP 200 on a test POST	Check host firewall and port 8000
6	Facility reset before the audience arrives	6 available, 0 sessions	Reset again and reload the client
7	Browser shows LINKED in the header	WebSocket connected	Reload; check the API port
8	One booking and one walk-in plate prepared	Both plates known to the operators	Prepare before the demo, not during
9	Manual release path rehearsed	Operator can free a bay in under a minute	Re-read Section 11.6
10	Telemetry panel still shows NOT MEASURED	No estimate is displayed	Do not fill it with guesses

Table 27 : Bench checklist before every demonstration

A.5 Troubleshooting guide

Table 28 collects the faults encountered on the bench, with the symptom that was actually observed and the cause that was actually found. It is written for the next person who inherits the rig.

Symptom	Most likely cause	Action
Board resets when the arm moves; serial log restarts	Servo drawing stall current from the board regulator	Separate 5 V supply, common ground, 470–1000 μF across the rail
Node appears dead; no serial output	Not in flash mode; GPIO 0 not held low at boot	Hold GPIO 0 to ground, reset, then release
Camera initialisation fails intermittently	Insufficient current or a loose camera header	Reseat the header; power from a stable adapter
All entries return `no_plate`	EasyOCR not installed or the plate model missing	Check `/health` model flags; install the engine
Vehicle admitted twice	Two frames arrived outside the cooldown window with different readings	Confirm the cooldown setting; check clock skew
Bay stuck occupied after a vehicle left	Exit plate unreadable; session still active	Operator manual release; record the plate
Dashboard shows stale bays	WebSocket dropped and the page was not reloaded	Reload; the client re-reads a snapshot on reconnect
Booking accepted but no bay named	Requested bay was taken; allocator substituted the first free bay	Expected behaviour; the response names the substitute
Receipts page blank	No completed sessions for the signed-in account	Complete an exit or sign in as the driver who did
Auth fails on a fresh checkout	Server pointed at the stale root database (D-09)	Delete the stale file and restart

Table 28 : Troubleshooting guide

PROGRAMME OUTCOMES

PO and PSO Mapping for Project Work

The project is mapped against the twelve programme outcomes and the three programme-specific outcomes of the Computer Science and Engineering (IoT, Cyber Security with Blockchain Technology) programme. The mapping strength records how much of the outcome the project actually exercises: *substantial* where the project is the primary evidence, *moderate* where it contributes a clear part, and *limited* where the exposure is real but narrow.

PO/PSO	Outcome	How the project addresses it	Strength
PO1	Engineering knowledge	Applies computer vision, databases, networks, embedded systems and web engineering to one problem	Substantial
PO2	Problem analysis	Occupancy sensing cost analysed from the literature; the boundary-only observation model derived and justified	Substantial
PO3	Design and development of solutions	Five-table schema, state machines, refusal policy, fail-closed actuation, eight-page client	Substantial
PO4	Conduct investigations	Measurement protocol defined with sampling, volume, metrics and honesty rule; two diagnostic records analysed	Moderate
PO5	Modern tool usage	FastAPI, Next.js, SQLAlchemy, OpenCV, YOLO11, EasyOCR, ESP32 toolchain, Git	Substantial
PO6	The engineer and society	Reduces search traffic and gate queueing; gate-only imaging limits surveillance inside the lot	Moderate
PO7	Environment and sustainability	Removes one powered device per bay; no trenching or cabling inside the facility	Moderate
PO8	Ethics	Refusal is recorded with evidence; privacy exposure stated; open defects disclosed rather than hidden	Substantial
PO9	Individual and team work	Work divided across recognition, backend, client, hardware and documentation, with shared review	Substantial
PO10	Communication	Architecture and sequence diagrams, interface documentation, this report and the demonstration script	Substantial
PO11	Project management and finance	Ten phases with deliverables, month-wise workflow, bill of materials and cost statement	Moderate
PO12	Life-long learning	Detector fine-tuning, OCR variant design, servo power engineering learnt within the project	Moderate
PSO1	IoT and embedded system development	Two ESP32-CAM nodes, servo actuation, power topology, firmware lifecycle	Substantial
PSO2	Secure and auditable software systems	JWT with bcrypt, role-aware operator actions, per-decision audit table	Moderate
PSO3	Applied analytics and modern tools	Object detection and OCR applied to a real decision, with diagnostics rather than a black box	Substantial

Table 29 : PO and PSO mapping

BUDGET

Project Cost

The hardware for this phase was drawn from department laboratory stock rather than purchased, and every software component is free or open source. No separate procurement was therefore made, and no cloud or licence cost was incurred. Table 30 records the position item by item so that the statement is complete rather than omitted; the quantities are the parts actually used, and the cost column states the basis rather than inventing an invoice value.

#	Item	Qty	Basis	Cost to the project
1	ESP32-CAM modules	2	Department laboratory stock	₹0 (stock)
2	SG90 micro-servos	2	Department laboratory stock	₹0 (stock)
3	5 V adapters and capacitors	2 each	Department laboratory stock	₹0 (stock)
4	USB-TTL programmer	1	Department laboratory stock	₹0 (stock)
5	Buzzer, jumpers, mounting	1 set	Consumables from stock	Not separately accounted
6	Host laptop	1	Already owned by the team	₹0
7	Wi-Fi access point	1	Existing institutional infrastructure	₹0
8	Python, Node.js, OpenCV, FastAPI, Next.js, SQLite	—	Open source	₹0
9	YOLO11 and EasyOCR	—	Open source; weights trained locally	₹0
10	Cloud hosting, database, licences	—	Not used — LAN deployment	₹0
	Total procurement for this phase		**Laboratory stock and open source**	**₹0**

Table 30 : Project cost estimation

The cost that the project actually demonstrates is not its own: it is the marginal sensing cost of a facility. A conventional design needs one sensing device, one mounting point and one power or data drop per bay; this design needs two cameras for the whole lot. At $n = 6$ the difference is four devices, and it grows by one device per bay thereafter. The report states that structural comparison rather than a purchase total, because the structural comparison is the claim the project is making.

PUBLICATION

Papers Published

No paper has been published from this work at the time of submission, and none has been submitted to a journal or conference. The material that would form the basis of a paper is the gate-only occupancy model and the refusal-by-default authorisation policy described in Chapters 5, 7 and 12; the measurement protocol of Section 10.5 is the outstanding prerequisite, because a paper on a decision policy under uncertainty requires the decision error rates that this report explicitly does not claim.

Sl. No.	Title of the paper	Journal / conference	Volume, issue, pages	Year
1	Nil — no publication from this work to date	—	—	—

If a paper is published before the final submission, this page should be replaced with the full bibliographic entry and the DOI, and the same entry added to the reference list.

COMPETITIONS

Project Competitions Participated and Won

The project was not entered in any external competition, hackathon or project exhibition during the two semesters of its development, and no award has been received. The demonstration script, the interface and the simulator were built to make such an entry straightforward: the system can be shown end to end on a laptop in under ten minutes, with the booked-entry, walk-in, refusal and exit paths each visible in a single sequence.

Sl. No.	Competition / event	Organiser	Level	Result	Year
1	Nil — no competition entered to date	—	—	—	—

FUNDING

KSCST or Other Funding Applied and Grants Received

The project did not apply for funding from the Karnataka State Council for Science and Technology (KSCST) or from any other agency, and no grant was received. The work was carried out with department laboratory stock and open-source software, as recorded on the project cost page, and at no direct procurement cost to the team.

The natural target for a future application is the KSCST Student Project Programme, for which the present work is well placed: the problem is local, the hardware cost is low, the design is unusual for its cost class, and the evaluation plan is already written. An application would need the bench measurements of H1–H7, a site, and a statement of the deployment partner; all three are identified in Chapter 15 as immediate next steps.

Sl. No.	Funding agency / scheme	Title of the proposal	Amount applied	Status	Year
1	KSCST Student Project Programme	Not applied	—	Nil	—
2	Any other agency	Not applied	—	Nil	—

INTELLECTUAL PROPERTY

Patents Filed and Published

No patent has been filed or published from this work. The novel element, if one is pursued, is the combination of gate-only occupancy inference with a refusal-by-default authorisation policy and a per-decision audit record for a small parking facility; the individual components — plate detection, OCR, servo barriers, booking systems — are all well established and would not by themselves support a claim.

Sl. No.	Title of the invention	Application number	Filed on	Status	Year
1	Nil — no patent filed to date	—	—	—	—

If a patent is filed before the final submission, this page should record the application number, the filing date and the current status, and a copy of the acknowledgement should be attached to the appendix.